

บทที่ 7

มาตรฐาน OGSA & OGSi

ในการพัฒนาระบบกริดนั้นได้มีผู้พัฒนามากมาย จากหลายๆที่ ซึ่งแต่ละที่แม้จะมีแนวความคิดที่เหมือนกัน แต่ถ้าจะนำมาใช้แล้ว จะสามารถใช้งานเข้าด้วยกันได้หรือไม่ ด้วยเหตุนี้เองทำให้ต้องมีการกำหนดมาตรฐานขึ้นมา เพื่อให้สามารถใช้งานระบบกริดร่วมกันได้ขึ้นมา โดยกำหนดออกมาเป็น Open Grid Service Architecture (OGSA)

OGSA เปรียบเสมือนพิมพ์เขียวในการที่จะสร้างมิดเดิลแวร์ขึ้นมา หรือสามารถกล่าวได้ว่า OGSA เป็นโปรโตคอลพื้นฐานที่กำหนดวิธีการการติดต่อ, การส่งข้อมูล, การจัดเตรียมรีซอร์ส, การแชร์รีซอร์ส, การทำตารางเวลา และอื่นๆที่เกี่ยวข้อง โดยอาศัยเทคโนโลยีอยู่ 2 อย่างในการสร้างมันขึ้นมา คือ เครื่องมือ และ เว็บเซอร์วิส โดยสร้างขึ้นมาเป็น Open Grid Service Infrastructure (OGSI) ที่จะนำมาใช้งานจริง

OGSI จะเป็นวิธีการในการสร้างกริดเซอร์วิส ซึ่งจะหมายถึงการจัดการ การแลกเปลี่ยนข้อมูลระหว่าง ผู้ใช้งานกริด และ เซอร์วิส อื่นๆ หรืออาจพูดได้ว่า เป็น เว็บเซอร์วิสที่จะช่วยเพิ่มความสะดวกให้กับการทำระบบกริด

7.1 Open Grid Service Architecture

ภายในโครงสร้างไอทีที่ใช้ในธุรกิจนั้น จะใช้ ส่วนจัดเตรียมเซอร์วิส (Service Provider) เป็นส่วนช่วยในการเพิ่มประสิทธิภาพให้โครงสร้างของระบบไอทีและการคำนวณแบบกริดจากหลายๆที่ โดยในการคำนวณนั้นจะเกี่ยวข้องกับการสร้าง การจัดการ และ แอปพลิเคชัน ที่เกิดจากการนำมารวมกันของ รีซอร์ส และ เซอร์วิส (และ ผู้ใช้งาน) ที่เรียกว่า Virtual Organizations ซึ่ง VO นั้นจะเป็นได้ทั้งแบบ เล็ก ใหญ่ มีอายุสั้น อายุยาว ใช้แบบเดียวใช้แบบหลาย และเป็นเนื้อเดียวกัน (homogeneous) หรือ เป็นแตกต่างกัน (heterogeneous)

Open Grid Service Infrastructure นั้นเป็นโครงสร้างที่ถูกกำหนดให้รองรับการสร้าง การดูแลรักษา และให้เป็น แอปพลิเคชันของเซอร์วิสที่ดูแลโดย VO's

7.1.1 Service Orientation and Virtualization

เมื่อกล่าวถึง VO จะมองรวมไปที่ รีซอร์สทางกายภาพ นั้นถูกใช้งานร่วมกันอยู่ หรือ อาจจะเป็น เซอร์วิส นั้นจะได้รับความช่วยเหลือจาก resource เหล่านั้น โดยใน OGSA ได้ให้คำจำกัดความของ เซอร์วิสว่ารีซอร์สที่ใช้ในการคำนวณ, รีซอร์สที่เป็นอุปกรณ์เก็บข้อมูล, ระบบเครือข่าย, โปรแกรม, ฐานข้อมูลและทุกสิ่งทุกอย่างที่คล้ายๆกับที่กล่าวมาว่าเป็นเซอร์วิส

Service-oriented จะทำให้สามารถกำหนดสิ่งที่ต้องการทั้งหมดเพื่อกำหนดมาตรฐานใน กลไกการทำงาน (mechanisms), ลักษณะการใช้งานแบบ local หรือ remote (local/remote transparency), การดัดแปลงให้เหมาะสมกับ OS services (adaptation to local OS services) และ ทำเซอร์วิสให้อยู่ในรูปแบบเดียวกัน (uniform service semantics) ซึ่ง Service-oriented นั้น จะทำให้ Virtualization ง่ายขึ้น ซึ่งเป็นความสามารถที่ encapsulation ภายใต้การ implement ในรูปแบบต่างๆ โดยที่ Virtualization ของกริดเซอร์วิสจะทำให้สามารถ map เซอร์วิสพื้นฐานได้สะดวกบนแต่ละ แพลตฟอร์ม

Virtualization จะง่าย ถ้า ฟังก์ชันของเซอร์วิสสามารถแสดงได้ในรูปแบบมาตรฐาน ดังนั้น ในการ สร้างเซอร์วิส ควรจะถูก invoke ด้วยวิธีเดียวกัน โดย WSDL ถูกนำมาใช้ในกรณีนี้ โดย WSDL จะให้ทำ multiple bindings สำหรับรูปแบบเดียวได้ โดยรวมถึงการกระจายโปรโตคอลสื่อสาร (เช่น HTTP) ได้ดี เช่นเดียวกับ locally optimized binding (เช่น local IPC) สำหรับการตอบโต้กันระหว่างการร้องขอและ กระบวนการต่างๆของเซอร์วิสบนเครื่องเดียวกัน

7.1.2 Service Semantics: The Grid Services

ในการที่จะทำให้เกิดมาตรฐานในการตอบโต้ของ เซอร์วิสได้นั้น จะต้องมีเซอร์วิสที่ต่างกันและใช้ในการแจ้งความผิดพลาดขึ้น ซึ่ง OGSA จะเป็นตัวที่กำหนด กริดเซอร์วิส ขึ้น ซึ่ง กริดเซอร์วิส นั้นคือเว็บเซอร์วิสซึ่งจัดเตรียมกลุ่มของรูปแบบที่กำหนดไว้และทำตามแบบแผนโดยเฉพาะโดยที่รูปแบบนี้จะทำให้ ค้นหาที่อยู่, สร้างเซอร์วิสแบบไดนามิก, จัดการไลฟ์ไทม์, ตรวจสอบ และควบคุมได้ง่าย

รูปแบบและแบบแผนที่กำหนด กริดเซอร์วิส นั้นย่อมต้องเกี่ยวข้องกัน โดยเฉพาะส่วนที่เกี่ยวข้องกันกับการจัดการ เซอร์วิสอินสแตนซ์แบบชั่วคราว ยกตัวอย่างเช่น เซอร์วิสอินสแตนซ์ อาจจะเป็นส่วนที่ กิวรีฐานข้อมูล, ทำเหมืองข้อมูล, จองแบนด์วิธของระบบเครือข่าย, ขนย้ายข้อมูล หรือรองรับความสามารถในการประมวลผล เป็นต้น โดยแบบชั่วคราวนั้นจะเป็นส่วนที่แสดงว่า เซอร์วิสจะถูก จัดการ, มีชื่อ, ค้นหา และใช้งานได้อย่างไร

7.1.2.1 Upgradeability Conventions and Transport Protocols

เซอร์วิสภายในระบบนี้ต้องมีความสามารถที่จะอัปเดตได้ โดยที่ไม่ทำให้ไคลเอนท์ที่ทำงานอยู่เกิดปัญหาขึ้น เช่น การ อัปเดตเครื่องที่ใช้ทำระบบนั้น อาจทำให้โปรโตคอลของระบบเครือข่ายเปลี่ยน โดย OGSA ได้กำหนดโครงสร้างที่จะทำการรีเฟรชให้ไคลเอนท์รู้เกี่ยวกับเซอร์วิส ที่มีการอัปเดตขึ้น เช่น บอกว่างานใดที่รองรับ หรือ บอกว่า โปรโตคอลเครือข่ายใดที่รองรับกับเซอร์วิสนั้นๆ เป็นต้น โดยที่เซอร์วิสนั้น จะต้องมีการ 2 คุณสมบัติที่สำคัญ ได้แก่

- Reliable service invocation คือ เซอร์วิสจะตอบโต้กับเซอร์วิสอื่นๆโดยการแลกเปลี่ยนข้อความในระบบแบบกระจายนั้น จะต้องมีความแน่นอนว่า ข้อความนั้นส่งไปถึงที่หมายได้จริงหรือไม่
- Authentication แต่ละเซอร์วิสนั้นจะต้องเป็นไปตามนโยบายที่กำหนดไว้ เช่น มีการตรวจสอบไคลเอนท์หรือตรวจสอบเซอร์วิสอินสแตนซ์ก่อนใช้งาน

7.1.2.2 Standard Interfaces

Interfaces ที่ใช้กำหนดกริดเซอร์วิสนั้นได้ถูกแสดงไว้ดังตารางด้านล่าง

PortType	Operation	Description
GridService	FindServiceData	Query a variety of information about the Grid service instance, including basic introspection information (handle, reference, primary key, home handleMap: terms to be defined), richer per-interface information, and service-specific information (e.g., service instances known to a registry). Extensible support for various query languages.
	SetTerminationTime	Set (and get) termination time for Grid service instance
	Destroy	Terminate Grid service instance
Notification-Source	SubscribeTo-NotificationTopic	Subscribe to notifications of service-related events, based on message type and interest statement. Allows for delivery via third party messaging services.
Notification-Sink	DeliverNotification	Carry out asynchronous delivery of notification messages
Registry	RegisterService	Conduct soft-state registration of Grid service handles
	UnregisterService	Deregister a Grid service handle
Factory	CreateService	Create new Grid service instance
HandleMap	FindByHandle	Return Grid Service Reference currently associated with supplied Grid Service Handle

ตารางที่ 7-1 อินเตอร์เฟซมาตรฐานของกริดเซอร์วิส

แอปพลิเคชันนั้นจะต้องการกลไกที่จะใช้ค้นหาเซอร์วิสที่ใช้งานได้ และสำหรับตัดสินใจเลือกลักษณะพิเศษของเซอร์วิส ที่จะใช้งานที่จะทำงานได้ตามที่ร้องขอ ตามนี้

- มาตรฐานที่ใช้แสดงข้อมูลของเซอร์วิส เป็นการแสดงข้อมูลเกี่ยวกับ กริดเซอร์วิสอินสแตนซ์ (Grid Services Instances) ซึ่งมีโครงสร้างเป็นกลุ่มของชื่อและอยู่ในรูปแบบ XML ซึ่งเรียกว่า Service data elements ซึ่ง encapsulate เป็นรูปแบบมาตรฐานไว้

- มาตรฐานของคำสั่งต่างๆ เช่น FindServiceData ที่ใช้สำหรับนำข้อมูลจาก กริดเซอร์วิสอินสแตนซ์ กลับมาใช้งาน

- มาตรฐานของการลงทะเบียนข่าวสารเกี่ยวกับ กริดเซอร์วิส ด้วย Registry Services และทำการ map จาก handle ให้เป็น reference จะกล่าวในหัวข้อต่อไป

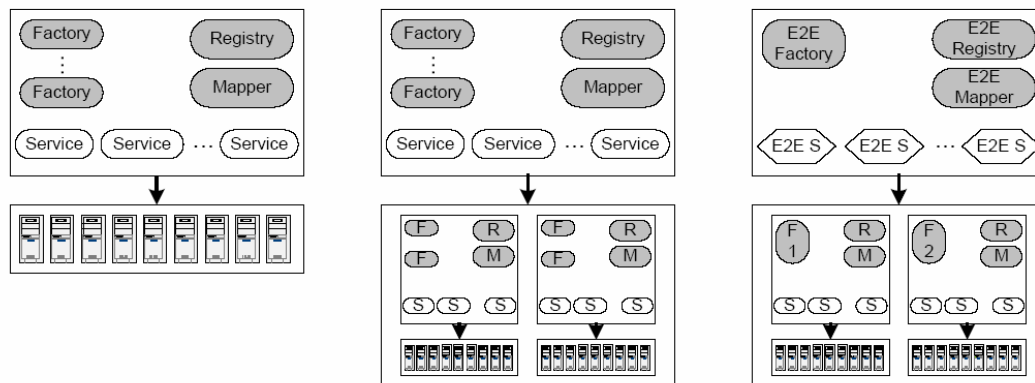
7.1.3 The Role of Hosting Environment

OGSA ได้กำหนดเกี่ยวกับ กริดเซอร์วิสอินสแตนซ์ ไว้ กล่าวคือ กำหนดว่า สร้างขึ้นมาอย่างไรให้ชื่อได้อย่างไร มีวัฏจักรชีวิตนานเท่าไร ติดต่อกับส่วนใดบ้าง เป็นต้น อย่างไรก็ตาม OGSA บอกในส่วนที่เป็นพฤติกรรมเบื้องต้น มันไม่ได้บอกว่าเซอร์วิสทำอะไร และไม่ได้บอกว่าทำเซอร์วิสขึ้นมา

อย่างไร เพราะ OGSA ไม่ได้บ่งบอกเกี่ยวกับ Programming Model, Programming Language, Implementation Tool หรือ Execution Environment เลย แต่ในปัจจุบันก็มีการใช้ภาษาหลากหลายรูปแบบในการสร้างเซอร์วิสเช่น C, C++, Java or Fortran

7.1.4 Using OGSA Mechanisms to Build VO Structures

แอปพลิเคชันและผู้ใช้งาน ต้องสามารถที่จะสร้างเซอร์วิสชั่วคราว และ ค้นหาและเลือกใช้ตามคุณสมบัติได้ ใน OGSA จะมี Factory, Registry, GridService and HandleMap ที่เป็นอินเทอร์เฟซที่รองรับการสร้างเซอร์วิสอินสแตนซ์ชั่วคราวและ ค้นหาและเลือกเซอร์วิสอินสแตนซ์ที่เกี่ยวข้องกับ VO ได้



รูปที่ 7-1 ตัวอย่าง 3 รูปแบบที่ใช้ใช้งาน

1. Simple Hosting Environment เป็น set ของ รีซอร์ส ที่ตั้งใน single administrative domain และ รองรับการจัดการเซอร์วิสโดยแต่ละ factory จะถูกบันทึกในรูปแบบ registry เพื่อให้ไคลเอนท์สามารถค้นพบ factories ที่ใช้งานได้ เมื่อมี factory ที่ถูกไคลเอนท์ร้องขอเพื่อสร้าง กริดเซอร์วิสอินสแตนซ์ แล้ว factory จะ invoke host-environment ให้สร้างอินสแตนซ์ใหม่ขึ้น และใส่ค่า handle และ register instance นั้น และสร้าง handle ที่ใช้งานได้ลงใน handleMap โดยลักษณะแบบนี้กล่าวได้อีกอย่างว่าเป็น Local Operations

2. Virtual Hosting Environment เช่นเดียวกับแบบแรก แต่จะเป็นการกระจายไปสู่หลายๆ hosting environment โดยที่ client ที่จะใช้งาน จะต้อง register ใน VO ก่อน เพื่อใช้ในการค้นหาและเรียกใช้งาน เป็นการจำลองการทำงานของ 2 environment ที่ต่างกัน แต่ใช้ทำงานร่วมกัน

3. Collective Services เปรียบเสมือนกับการสร้าง Virtual Hosting Environment หลายๆอัน โดยที่แต่ละส่วนนั้นจะเป็นเซอร์วิสอินสแตนซ์ระดับสูง (High Level Service Instance) ต่างกับแบบที่ 2 ที่จะเป็นเพียงเซอร์วิสอินสแตนซ์แบบธรรมดาโดยเซอร์วิสอินสแตนซ์ระดับสูงนั้นจะเปรียบเสมือน เซอร์วิสอินสแตนซ์หลายๆอันที่รวมกันอยู่

7.2 รายละเอียดทางเทคนิคของ OGSA

ในส่วนนี้จะอธิบายรายละเอียดเกี่ยวกับ กริดเซอร์วิส แบบลึกๆ และความเกี่ยวข้องกับรูปแบบตามข้อตกลง

7.2.1 OGSA Service Model

ในส่วนของ OGSA นั้นกล่าวได้ว่าเป็นส่วนที่กล่าวถึงเซอร์วิสซึ่งเป็น entity ทางเครือข่ายที่จัดเตรียมความสามารถในการแลกเปลี่ยนข้อความ อันได้แก่รีซอร์สที่ใช้ในการคำนวณ, รีซอร์สทางกายภาพ, ระบบเครือข่าย, โปรแกรม, ฐานข้อมูล และ เซอร์วิสทั้งหมด โดยที่ OGSA จะแสดงทุกอย่างตามที่กล่าวมาให้เป็น กริดเซอร์วิส

กริดเซอร์วิส เป็นการแสดงลักษณะที่มันสามารถทำได้ ซึ่งจะ implement ออกมาเป็น 1 หรือหลายๆอินเตอร์เฟซ โดยที่แต่ละอินเตอร์เฟสนั้นจะกำหนดกลุ่มของกระบวนการที่จะ invoke โดยการแลกเปลี่ยนการกำหนดลำดับของข้อความ ซึ่ง กริดเซอร์วิส จะตอบโต้กันผ่านทาง portTypes ของ WSDL ซึ่งใน set ของแต่ละ portTypes นั้นจะถูกรองรับด้วย กริดเซอร์วิส

กริดเซอร์วิส จะมีการดูแลสแตทภายในของมัน สำหรับการจัดการไลฟ์ไทม์ของเซอร์วิส ซึ่งช่วงที่คงอยู่ของสแตทนั้นจะแบ่งแยกออกเป็น 1 อินสแตนซ์ของเซอร์วิสซึ่งจะใช้กริดเซอร์วิสอินสแตนซ์เป็นส่วนอ้างอิงถึงกริดเซอร์วิส

การผูกโปรโตคอลที่เกี่ยวข้องกับเซอร์วิสที่ส่งออกไปนั้น เช่น เซอร์วิสมีการตอบโต้กับอีกเซอร์วิส โดยการแลกเปลี่ยนเมสเสจสิ่งที่ไม่ได้ต้องการให้ในระบบการคำนวณแบบกระจายนั้น จะถือว่าเป็นการล้มเหลว อย่างไรก็ตามส่วนนั้นไม่สามารถรับรองได้ว่าได้ถูกส่งจริง แต่จะใช้การคงอยู่ของสแตทภายในของมันเป็นตัวรับรองว่าเซอร์วิสได้รับข้อความหรือยัง แม้ว่าจะต้องใช้การส่งใหม่อีกครั้งถ้ายังไม่ได้รับก็ตาม

OGSA เซอร์วิสจะถูกสร้างหรือถูกทำลายแบบ dynamic ซึ่งบางที เซอร์วิสอาจจะถูกทำลายจริงหรือไม่สามารถเข้าถึงระบบได้เนื่องจากเหตุการณ์หลายๆกรณี ไม่ว่าจะเป็นกรณีผลลัพธ์ของระบบผิดพลาด, ระบบปฏิบัติการล่มหรือ ระบบเครือข่ายล่มทำให้ต้องมีอินเตอร์เฟซที่จะกำหนดการจัดการเกี่ยวกับไลฟ์ไทม์

เนื่องจาก กริดเซอร์วิส เป็นแบบ dynamic และ stateful ทำให้มันจะต้องมีทางที่จะแยกการสร้างเซอร์วิสอินสแตนซ์ จากอันอื่นๆที่สามารถสร้างขึ้นมาได้เหมือนกัน ในทุกๆ กริดเซอร์วิสอินสแตนซ์จะต้องมี Global Unique Name ที่เรียกว่า Grid Service Handle (GSH) ที่จะใช้แยก กริดเซอร์วิสอินสแตนซ์ จากอันอื่นๆที่ยังคงอยู่ ไม่ว่าในปัจจุบัน หรือในอนาคตที่กำลังจะมีอันใหม่เพิ่มขึ้นมาอีก (ในกรณีที่ล้มเหลว แล้วมีการสร้างอินสแตนซ์อันเดิม ขึ้นมาใหม่ จะใช้ GSH อันเดิมที่เคยใช้งาน)

ซึ่งกริดเซอร์วิสนั้นจะสามารถอัพเกรด ได้ในระหว่างที่ยังมีชีวิตอยู่ เช่น เมื่อมีการใช้โปรโตคอล เวอร์ชันใหม่ หรือ เพิ่มโปรโตคอล อื่นๆเข้าไป ด้วยเหตุนี้ทำให้ GSH จะไม่อยู่บนโปรโตคอล หรืออินสแตนซ์จะไม่มีข้อมูลที่เจาะจง เช่น ที่อยู่บนเครือข่ายหรือโปรโตคอลที่รองรับ โดยข้อมูลนี้จะถูก encapsulate กับ อินสแตนซ์ อื่นๆที่ใช้ระบุเจาะจงเกี่ยวกับการตอบโต้กับ เซอร์วิสอินสแตนซ์ ที่เรียกว่า

Grid Service Reference (GSR) ที่แตกต่างกับ GSH โดย กริดเซอร์วิสอินสแตนซ์ จะเปลี่ยนได้กระทั่ง Service Lifetime เนื่องจาก GSR จะมีเวลาหมดอายุอย่างตรงๆ หรือแม้กระทั่งจะยกเลิกได้แม้กระทั่งในไลฟ์ไทม์ และ OGSA จะกำหนดการ mapping สำหรับการอัปเดต GSR

อย่างที่บอกไว้ใน OGSA นั้นทุกอย่างคือเป็น กริดเซอร์วิส ซึ่งทำให้จะต้องมี กริดเซอร์วิส ที่ใช้ในการสร้าง กริดเซอร์วิส, handle และ reference ที่ได้กำหนดไว้ใน OGSA model

7.2.2 Creating Transient Services: Factories

OGSA ได้กำหนด class ของ กริดเซอร์วิส ที่ใช้สร้าง กริดเซอร์วิสอินสแตนซ์ ซึ่งเราเรียกว่า Factory ซึ่งใน Factory interface's CreateService จะเป็น operation ที่ใช้สร้างการร้องขอ กริดเซอร์วิส และส่งค่า GSH และค่าเริ่มต้นของ GSR ไปให้เซอร์วิสอินสแตนซ์ใหม่

ใน Factory interface นั้น ไม่ได้ระบุว่า เซอร์วิสอินสแตนซ์ สร้างขึ้นมาได้อย่างไร เนื่องจาก factory interface จะถูก implement บางส่วนจาก hosting environment (เช่น .NET หรือ J2EE) ที่มีกลวิธีในการสร้าง Service instance ใหม่ๆ ซึ่ง hosting environment จะกำหนดว่า เซอร์วิสจะสร้างได้อย่างไร (เช่น ใช้ภาษาอะไรเขียน เป็นต้น) โดยที่การสร้าง factory ที่มีระดับสูงขึ้นไปนั้น อาจจะใช้การสร้างจากการรวมกันของหลายๆ factory

7.2.3 Service Lifetime Management

ในการจัดการไลฟ์ไทม์ นั้น มีอยู่ 2 กรณี คือ

- โคล์เอนท์ทราบหรือเป็นตัวตัดสินใจที่จะให้กริดเซอร์วิสนั้นถูกยกเลิกไป ส่วนนี้เป็นส่วนที่จะทำให้โคล์เอนท์แน่ใจที่จะได้รับข่าวสารว่า เซอร์วิสอินสแตนซ์ นั้นได้ถูกยกเลิกไปแล้ว และ รีซอร์ส ที่ใช้ไปได้กลับคืนมาแล้ว แม้ว่าจะเกิดจากกรณีที่ระบบล้มเหลว (เช่น เกิดจากการล่มของเซิร์ฟเวอร์, เครือข่าย หรือ โคล์เอนท์) ซึ่งโคล์เอนท์จะต้องทราบสถานะสุดท้ายของ เซอร์วิสอินสแตนซ์ หรือ การร้องขอนั้น ถ้าเป็นในกรณีที่ระบบล่มจะต้องทราบว่ามันจะต้องติดต่อ เซอร์วิส ไหนต่อหลังจากที่มันยกเลิกไปแล้ว โดยที่ทรัพยากรที่เกี่ยวข้องกับ เซอร์วิสนั้นๆจะถูกปลดปล่อยหลังจากเวลานั้น และอย่างน้อยโคล์เอนท์จะต้องสามารถขยายไลฟ์ไทม์ได้ด้วย
- Hosting Environment เป็นตัวรับประกันว่ารีซอร์สนั้นเป็นส่วนที่สิ้นเปลือง แม้ว่าจะเกิดจากกรณีการผิดพลาดนอกการควบคุม ก็ตาม ถ้าเวลาที่ควรยกเลิกเซอร์วิสมาถึง hosting environment นั้นจะต้องสามารถที่จะเรียกรีซอร์สที่เกี่ยวข้องคืน

ซึ่งกลวิธีของมันจะแบ่งออกเป็นรอบๆ ตามนี้

- Negotiation an initial lifetime เป็นช่วงที่มีการร้องขอที่จะสร้าง กริดเซอร์วิสอินสแตนซ์ ใหม่ผ่าน factory client จะต้องเป็นตัวแสดงเวลาที่น้อยที่สุดและมากที่สุดที่จะยอมรับได้ในการตั้ง initial lifetime และ factory จะเป็นตัวเลือกค่า initial lifetime นั้นส่งกลับไปให้โคล์เอนท์

- Request a lifetime extension เป็นช่วงที่ไคลเอนท์จะร้องขอให้ขยายไลฟ์ไทม์ผ่านข้อความที่ชื่อว่า SetTerminalTime เข้าไปให้กริดเซอร์วิสอินสแตนซ์ ซึ่งจะระบุเวลาที่ยอมรับได้ที่น้อยและมากที่สุดไปให้

7.2.4 Managing Handles and Reference

จากที่กล่าวไปในช่วงต้น การตอบของ Factory นั้น จะตอบเป็น GSH และ GSR โดยที่ GSH เป็นการรับรองการอ้างถึงการสร้างเซอร์วิสอินสแตนซ์ และ GSR จะถูกสร้างภายในช่วงเวลาจำกัด และจะเปลี่ยนได้ในระยะเวลาของ service lifetime การที่ GSR มันสามารถเปลี่ยนแปลงได้นั้นอาจทำให้เกิดผลพลาดได้ ซึ่งจะต้องมีวิธีการที่จะใช้เพียง GSH เพื่อนำมาใช้หาค่า GSR ที่ถูกต้องได้

วิธีการนั้นก็คือ HandleMap ซึ่งเป็นกระบวนการที่จะรับค่า GSH และตอบกลับเป็น GSR ที่ถูกต้อง กระบวนการ map ของ HandleMap นั้น จะเก็บ track ว่า กริดเซอร์วิสอินสแตนซ์ แท้จริงแล้วยังมีชีวิตอยู่มั้ย และไม่ตอบรับค่าอินสแตนซ์ ว่ามันต้องยกเลิกอย่างไร อย่างไรก็ตาม การมีของ GSR ที่ถูกต้อง ไม่ได้เป็นการแน่นอนว่า กริดเซอร์วิสอินสแตนซ์ นั้นจะติดต่ออย่างไร และ เซอร์วิสบางที่จะล่มหรือ ยกเลิกระหว่างเวลาที่ GSR ไม่ได้ให้ออกมาว่าถูกใช้อยู่ เกี่ยวกับ HandleMap Interface นั้นจะมีปัญหาที่เกี่ยวกับบรรจุ GSR เพื่อให้เป็นเซอร์วิสอยู่ 2 ปัญหา ดังนี้

- 1) การบอก handleMap service ว่ามันบรรจุการ map จาก GSH ที่เจาะจง
- 2) การติดต่อกับ handleMap เพื่อให้บรรจุค่า GSR ที่ต้องการเข้าไป

ใน 2 ปัญหานี้ เป็นปัญหาที่เกี่ยวกับการ map GSH เป็น GSR ซึ่งจำเป็นต้องให้ทุกๆกริดเซอร์วิสอินสแตนซ์ จะต้องถูกลงทะเบียนอย่างน้อย 1 ครั้งจาก handleMap ซึ่งเรียกว่า home handleMap โดยโครงสร้างแล้ว GSH จะรวม home handleMap's identity เข้าไปด้วย โดยการติดต่อให้ handleMap บรรจุค่า GSR จากค่า GSH ที่ให้ไป สามารถระบุและกำหนดได้โดยง่ายเนื่องจากว่า มีชื่อที่เป็นเอกลักษณ์ในแต่ละ local นั้นๆ เพื่อหลีกเลี่ยงการซ้ำกันของ service ที่ถูกสร้างขึ้นมาจากศูนย์กลาง แม้ว่าอาศัย Domain Name System อย่างไรก็ตามทุกๆ GSH นั้นจะต้องมี 1 home handleMap

ในการสร้าง HandleMap นั้น เป็นการใช้กริดเซอร์วิสสร้าง และมันจะต้องมี GSH ให้ จึงใช้ค่า GSH นี้ในการสร้าง ซึ่งจะทำให้ได้ข้อต้องการ ให้ home handleMap ถูกกำหนดค่าโดย URL และรองรับ bootstrapping operation ที่จะทำให้เป็นหนึ่งเดียว โดยใช้โปรโตคอลที่รู้จักกันดีในการให้ชื่อ เช่น HTTP (หรือ HTTPS) เนื่องจากการใช้ GSR เพื่อบอกว่า โปรโตคอลอะไรที่ควรจะติดต่อกับ handleMap service ซึ่งการใช้ HTTP GET operation ที่ถูกใช้บน URL ที่จุดนั้นเพื่อให้เป็น home handleMap และ GSR สำหรับ handleMap ในรูปแบบ WSDL เพื่อตอบกลับไป

ความสัมพันธ์ระหว่างเซอร์วิส ที่ implement HandleMap และ Factory interface โดยเฉพาะ GSH จะ ตอบกลับโดย factory request ที่ต้องบรรจุ URL ของ home handleMap และ GSH/GSR เพื่อ mapping จะต้องเข้าไปและปรับปรุงค่าได้ใน handleMap service ในการ implement factory จะต้อง

ตัดสินใจว่าจะให้เซอร์วิสอะไรไว้ใน home handleMap ที่จริงแล้วเพียงเซอร์วิสเดียวบางทีจะสามารถสร้างได้ทั้ง Factory และ HandleMap Interface

7.2.5 Service Data และ Service Discovery

ส่วนที่เกี่ยวข้องกับกริดเซอร์วิสอินสแตนซ์นั้นเป็นกลุ่มของข้อมูลที่เรียกว่า service data ซึ่งเป็นกลุ่มของค่า XML ที่ฝังอยู่ในค่าของข้อมูลของเซอร์วิส ซึ่งในแต่ละค่านั้นจะมีชื่อของกริดเซอร์วิสอินสแตนซ์นั้น, ชนิด, เวลาที่ยังมีชีวิตอยู่ ซึ่งเป็นข้อมูลที่ใช้ในกระบวนการจัดการไลฟ์ไทม์

รูปแบบของกริดเซอร์วิสนั้นจะกำหนดด้วยคำสั่ง WSDL โดยใช้ FindServiceData สำหรับการควิรีและเลือกเอามาจากข้อมูลของเซอร์วิส ซึ่งคำสั่งนี้จะใช้ชื่อในการควิรี และอาจขยายไปรูปแบบอื่นๆ เช่น Xquery ที่ใช้กับกรณีพิเศษๆบางกรณี

คุณสมบัติของกริดเซอร์วิสนั้นจะต้องประกอบไปด้วยค่า service data อย่างน้อย 1 ค่าที่รองรับโดยกริดเซอร์วิสอินสแตนซ์ใดๆ ค่าที่ใช้ เช่น GSH,GSR, คีย์ชั้นปฐมภูมิ และค่า handleMap เป็นต้น

ใน 1 แอปพลิเคชันของกริดเซอร์วิสนั้นจะต้องมี FindService ที่ใช้ค้นหาเซอร์วิส ซึ่งจะใช้ค่า GSH ในการค้นหา และจะพิจารณาจากค่า การจัดเตรียม, จำนวนที่ร้องขอใช้เซอร์วิส, ค่าโหนดของเซอร์วิส, หรือนโยบายที่ใช้จัดการ มาใช้เลือกเซอร์วิสที่เหมาะสม

กริดเซอร์วิสที่รองรับการค้นหาจะถูกเรียกโดยค่า registry ซึ่งใช้กำหนดอยู่ 2 อย่าง คือรูปแบบการลงทะเบียน ที่จะกำหนดว่า GSH ใดที่จะให้ลงทะเบียนด้วยเซอร์วิสที่ใช้ลงทะเบียนและใช้กำหนดค่าที่เกี่ยวข้องกับการลงทะเบียนของ GSH

การลงทะเบียนนั้นเป็นการยอมให้ GSH ที่ลงทะเบียนไว้ด้วยเซอร์วิสที่ใช้ลงทะเบียนเพื่อคัดเลือกรุ่นย่อยให้ได้อย่างถูกต้อง ตรงตามความต้องการ เพื่อสะดวกต่อการค้นหาตามที่ร้องขอ ซึ่งค่าที่ลงทะเบียนของ GSH นั้นจะต้องรีเฟรชเป็นระยะๆ เพื่อให้เซอร์วิสที่ใช้ค้นหาสามารถเลือกเซอร์วิสที่เหมาะสมที่สุดได้

7.2.6 Notification

ใน OGSA ส่วนเฟรมเวิร์คของ notification นั้นจะเป็นส่วนที่ยอมให้ไคลเอนท์จะลงทะเบียนประกาศเฉพาะทางที่สำคัญได้ (ในรูปแบบ NotificationSource) เพื่อจะส่งไปให้ผู้รับตามที่ต้องการ (NotificationSink) ถ้าเซอร์วิสใดต้องการให้รองรับข้อความ notification แล้ว มันจะต้องรองรับรูปแบบของ NotificationSource ด้วยซึ่งเซอร์วิสที่ต้องการข้อความ notification จะต้องสร้าง รูปแบบ NotificationSink ขึ้นมาซึ่งใช้ในการส่งข้อความนั้นๆซึ่งขั้นตอนของมันจะเริ่มจากต้องรับรูปแบบของแหล่งข้อมูลก่อน แล้วให้ค่า GSH ของ sink ไปเพื่อใช้ส่งข้อความ โดยที่ sink จะต้องส่งข้อความที่บ่งบอกว่าสนใจข้อความ notification นั้นไปด้วย

สิ่งสำคัญในรูปแบบของ notification นั้นคือต้องปิดการรวมตัวกันด้วยข้อมูลของเซอร์วิส โดยมีคำสั่งพิเศษคือ “push” ที่ใช้ส่งข้อมูลตามเงื่อนไขพิเศษ (โดยการเรียกคำสั่ง FindServiceData เพื่อเตรียมที่จะ “pull”) ซึ่งเฟรมเวิร์คที่ใช้นี้จะยอมให้มีการส่งข้อความตรงๆจากเซอร์วิสไปยังเซอร์วิส รวม

ถึงการส่งไปยังเซอร์วิสกลุ่มที่ 3 ด้วย เช่น การส่งข้อความที่สำคัญทางธุรกิจไปให้กลุ่มที่ต้องการ เป็นต้น โดยต้องมีโปรโตคอลที่รองรับการส่งแบบมัลติคาสต์ด้วย

7.2.7 การควบคุมการเปลี่ยนแปลง

เพื่อรองรับการส่งและการควบคุมการเปลี่ยนแปลงของกริดเซอร์วิส ได้อย่างมีประสิทธิภาพ รูปแบบของกริดเซอร์วิสจะต้องเป็นแบบที่ทั่วถึงกัน (globally) และมีชื่อเป็นเอกเทศกัน (uniquely) ซึ่งใน WSDL นั้นจะกำหนดไว้ใน portType ด้วย qname โดยการเปลี่ยนแปลงใดๆจะต้องสร้างคำจำกัดความของเซอร์วิสที่เปลี่ยนแปลงนั้น ซึ่งการเปลี่ยนแปลงต้องรองรับกับรูปแบบเดิมด้วย โดยส่วนนี้จะเป็นส่วนที่ยอมให้ไคลเอนท์ที่ต้องการกริดเซอร์วิสนั้นๆด้วยคุณสมบัติพิเศษที่เหมาะสมกับงานของตน เพื่อให้เซอร์วิสนั้นๆใช้ได้อย่างมีประสิทธิภาพ

7.2.8 การผูกกับโปรโตคอลเครือข่าย

ในเฟรมเวิร์คของเว็บเซอร์วิสนั้นจะใช้โปรโตคอลที่แตกต่างกันมาร่วมกันทำงาน เช่น ใช้ SOAP+HTTP พร้อมด้วย TLS สำหรับด้านความปลอดภัย ส่วนนี้จะบ่งบอกว่า OGSA ใช้อะไรบ้าง ซึ่งจะแบ่งเป็น 4 ส่วนที่มีความสำคัญดังนี้

- Reliable transport: อย่างที่กล่าวไปแล้วว่ากริดเซอร์วิสนั้น จำเป็นจะต้องมีการส่งเซอร์วิสไปได้อย่างน่าเชื่อถือ ซึ่งโปรโตคอลที่รองรับด้านนี้ก็เช่น HTTP-R
- Authentication and delegation: จากที่กริดเซอร์วิสนั้นจำเป็นที่จะต้องรองรับการติดต่อสื่อสารไปยังที่อื่นๆ ซึ่งต้องมีการรับรองด้วย ทางหนึ่งที่ใช้กันคือ ใช้การสื่อสารแบบทางเดียว เช่น ใช้ TLS ช่วยรองรับด้านนี้
- Ubiquity: เมื่อเป้าหมายของกริดคือเพื่อให้เป็นแหล่งรวมของ VO จากริชอร์สที่กระจายตามส่วนต่างๆ จึงจำเป็นต้องมีการเลือกเซอร์วิสที่เหมาะสมที่จะใช้ตอบโต้กัน
- GSR Format: ในการเรียก GSR นั้น จะต้องมีการผูกกันด้วยรูปแบบพิเศษ 1 ในรูปแบบที่ใช้ก็คือ รูปแบบเอกสาร WSDL แต่ก็มีทางเลือกอื่น เช่น CORBA IOR เป็นต้น

7.2.9 Higher-Level Services

ส่วนทฤษฎีและเซอร์วิสที่กล่าวในหัวข้อนี้ จะบอกถึงการเตรียมสิ่งที่จะใช้สร้างกริดเซอร์วิสระดับสูงขึ้นไป ซึ่งเอาไปใช้ในด้าน ธุรกิจ หรือ วิทยาศาสตร์ ซึ่งจะอาศัยส่วนต่างๆ ต่อไปนี้

- Distributed data management service: เป็นส่วนที่รองรับการเข้าถึงและการย้ายของข้อมูลที่กระจายกัน ไม่ว่าจะเป็นฐานข้อมูลหรือเพิ่มข้อมูล ซึ่งเซอร์วิสที่รวมอยู่ในนี้จะมีการเข้าถึงฐานข้อมูล, การแปลงข้อมูล, การจำลองการจัดการ, การจำลองตำแหน่งที่เก็บ และ ทรานแซคชัน
- Workflow service: เป็นส่วนที่รองรับการทำงานของหลายๆแอปพลิเคชันบนริชอร์สที่กระจายกันอยู่
- Auditing service: เป็นส่วนที่รองรับการจำข้อมูล ทำให้ที่เก็บข้อมูลมีความปลอดภัย วิเคราะห์ถึงข้อมูลที่มีจุดประสงค์ไม่ดีหรือบุกรุกเข้ามา

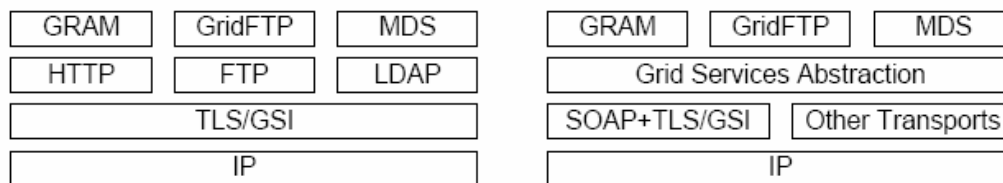
- Instrumentation and monitoring service: รองรับการค้นหาในสภาพแวดล้อมแบบกระจาย โดยจะอาศัย เซ็นเซอร์ช่วย ซึ่งจะมีหน้าที่รวบรวมข่าวสารและวิเคราะห์ เพื่อใช้เตือนสิ่งที่ผิดปกติต่าง ๆ

- Problem determination service for distributed computing: รวมหลายๆวิธีการ เช่น dump, trace และ log ด้วยการบันทึกเหตุการณ์และด้วยความสามารถในการเข้าคู่กัน

- Security protocol mapping services: จัดเตรียมโปรโตคอลที่ปลอดภัยสำหรับการกระจายส่วนต่างๆ โดยจะ map เข้ากับรูปแบบทั่วไป โดยมีการรับรองความปลอดภัยในการกระจายและการควบคุมสิทธิการเข้าถึงส่วนต่างๆ

ความยืดหยุ่นของเฟรมเวิร์คนี้ก็คือ สามารถสร้างและรวมได้จากหลากหลายวิธี เช่น รวมเซอร์วิสจากส่วนที่จองไว้และใช้หลายๆรีซอร์สมาช่วยในการคำนวณ หรือ อาจจะเชื่อมกับแอปพลิเคชันที่เป็นไลบรารี หรือ รวมกับเซอร์วิสระดับสูง เป็นต้น

ดังนั้น เพื่อความสะดวกในการจัดการรีซอร์ส, การขนส่งข้อมูล, การใช้โปรโตคอลของเซอร์วิสใน Globus Toolkit นั้นจึงสร้างเครื่องมือในด้านต่างๆไว้ ดังภาพด้านล่างนี้



รูปที่ 7-2 ภาพทางซ้ายเป็นโปรโตคอลของ Globus Toolkit 3 ส่วนภาพทางขวาเป็นแนวคิดของ OGSA

7.3 Open Grid Service Infrastructure

OGSI (Open Grid Services Infrastructures) นั้นเป็นส่วนที่คิดขึ้นมาเพื่อใช้กำหนดคุณสมบัติของระบบกริด ซึ่งจะกล่าวถึง ความสัมพันธ์ระหว่างโครงสร้างของกริด, ระบบการคำนวณแบบกระจาย, ความเกี่ยวข้องกับเว็บเซอร์วิส เฟรมเวิร์ค และความสัมพันธ์ระหว่าง client กับ hosting environment

7.3.1 ความสัมพันธ์กับระบบการคำนวณแบบกระจาย

กริดเซอร์วิส (Grid Service) นั้นจะสร้างขึ้นแบบฝังไว้ภายในแต่ละอินสแตนซ์ (instance) ซึ่งมีส่วนที่ใช้อธิบายอินสแตนซ์ ก็คือ WSDL portType โดยหลายๆ Grid Service นั้นจะรวมตัวกัน เรียกว่า Grid Services Factory ซึ่งใช้สร้างอินสแตนซ์ สำหรับแต่ละ portType โดยที่แต่ละอินสแตนซ์ จะเป็นส่วนที่ใช้ติดต่อกันระหว่าง 2 กริดเซอร์วิส ในการทำงานแต่ละครั้งก็จะติดต่อไปที่อินสแตนซ์ ให้ส่งค่าที่ต้องการกลับมา

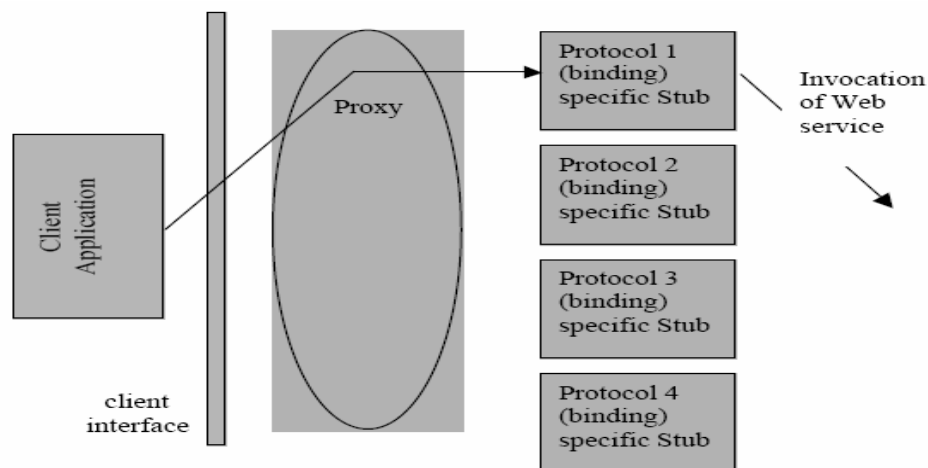
กริดเซอร์วิสอินสแตนซ์นั้น ใช้เพื่อให้เข้าถึงแอปพลิเคชันของไคลเอนต์โดยอาศัย Grid Service Handle และ Grid Service Reference เพื่อส่งรีเควสต์ตรงไปที่แต่ละอินสแตนซ์ที่เป็นเป้าหมายซึ่งจะได้

รับคำตอบเป็น เครื่องเป้าหมายที่จะส่งงานไปให้คำนวณ แสดงออกมาเป็นค่า Grid Service Reference ที่ใช้ในการติดต่อกันจริง

โดยสรุปแล้วสิ่งที่เรียกว่าระบบการคำนวณแบบกระจายในกริดนั้น จะอาศัย GSR ในการกำหนดว่าควรส่งงานไปให้ที่ส่วนไหนทำงานโดยอาศัยตัวจัดการที่เรียกว่าตัวจัดเตรียมเซอร์วิส (Service Provider) ที่เก็บรวบรวมข้อมูลว่า แต่ละโฮสต์ในกริด นั้นมีเซอร์วิสที่มีไคลเอนต์ที่ต้องการเรียกใช้หรือไม่ ถ้าเครื่องไหนมีและพบว่ามีความพร้อมที่สุด ก็จะส่งไปเครื่องนั้นๆทำงาน

7.3.2 แนวทางการเขียนโปรแกรมฝั่งไคลเอนต์

เนื่องจากกริดในยุคหลังนั้น ได้นำเทคโนโลยีของเว็บเซอร์วิส มาช่วยในการพัฒนาส่วนต่างๆ โดยส่วนที่ OSGI นำเว็บเซอร์วิสขึ้น ส่วนหนึ่งก็คือ WSDL ที่ใช้อธิบายว่า ผูกกับโปรโตคอลอย่างไร, เอนโค้ดอยู่ในรูปแบบใด, ส่งข้อความแบบใด (ระหว่าง RPC หรือ document-oriented) และใช้เว็บเซอร์วิส อย่างไร ซึ่งในกริด จะใช้ Web Service Invocation Framework ร่วมกับ Java API for XML RPC ในหลายๆส่วนของซอฟต์แวร์ ที่พัฒนาขึ้น

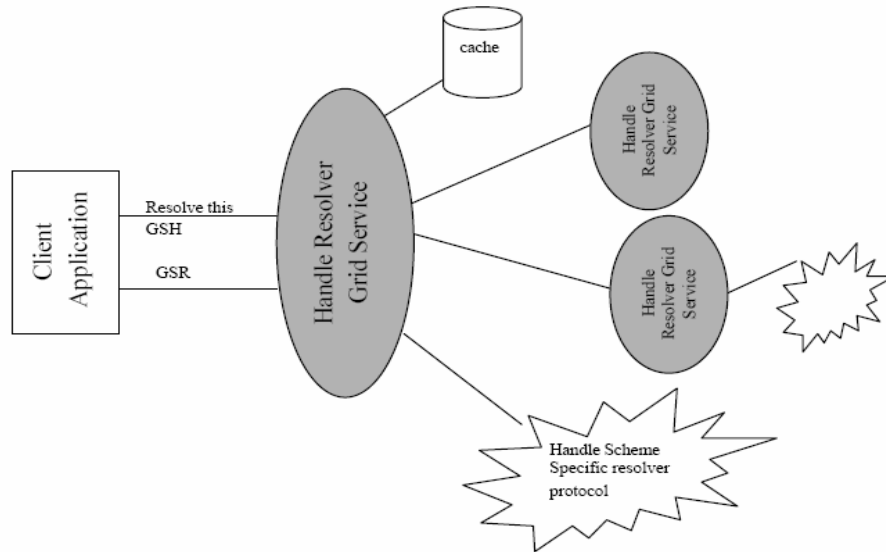


รูปที่ 7-3 แสดงโครงสร้างการทำงานฝั่งไคลเอนต์

7.3.3 การใช้ Grid Services Handles และ References ของไคลเอนต์

การที่ไคลเอนต์จะเข้าถึงกริดเซอร์วิสอินสแตนซ์ได้นั้น จะต้องผ่าน GSH (Grid Service Handle) และ GSR (Grid Service Reference) โดยที่ GSH นั้นจะเปรียบเสมือน Network Pointer ที่จะชี้ไปที่กริดเซอร์วิสอินสแตนซ์ แต่ GSH ไม่ใช่ส่วนที่จัดเตรียมข้อมูลที่จะให้ไคลเอนต์เข้าถึงเซอร์วิสอินสแตนซ์ได้ การที่จะเข้าถึง กริดเซอร์วิสอินสแตนซ์ได้นั้น ไคลเอนต์จะต้องผ่านวิธีการแปลง GSH ให้เป็น GSR (Grid Service Reference) ซึ่ง GSR จะเป็นส่วนเก็บข้อมูลที่จำเป็นสำหรับการเข้าถึงกริดเซอร์วิสอินสแตนซ์ ไว้ โดยที่ GSR ไม่ใช่ Network Pointer ที่ชี้ไปที่กริดเซอร์วิสอินสแตนซ์ เนื่องจาก GSR อาจทำให้ผิดพลาดได้ในบางกรณี เช่น กริดเซอร์วิสอินสแตนซ์ นั้นย้ายไปที่ Server ตัวอื่นแล้ว เป็นต้น

OGSI นั้นได้จัดเตรียมวิธีการที่จะทำ HandleResolver เพื่อรองรับให้ไคลเอนต์สามารถแปลง GSH ให้เป็น GSR ได้ ในรูปด้านล่างจะแสดงวิธีการให้ดู

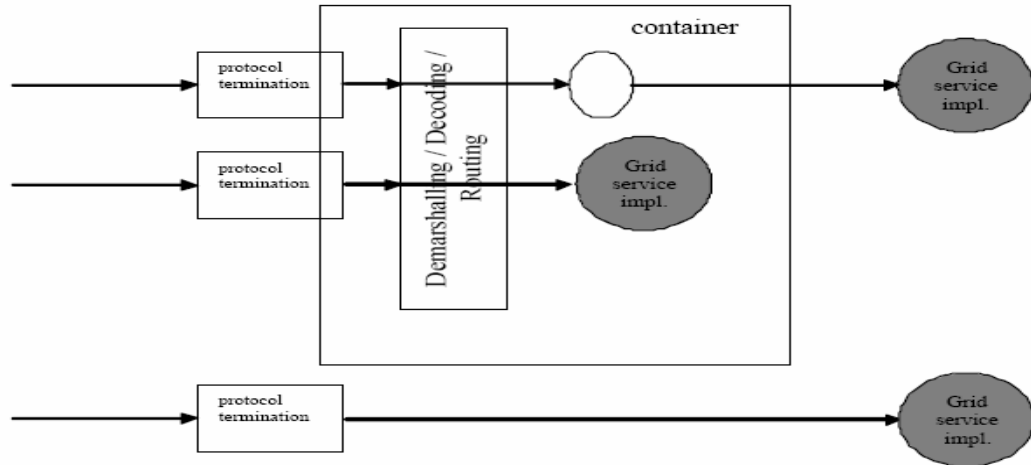


รูปที่ 7-4 แสดงวิธีการ resolve GSH

ไคลเอนต์นั้นจะแปลงจาก GSH เป็น GSR โดยผ่านการ invoke ที่ HandleResolver กริดเซอร์วิสอินสแตนซ์โดยที่ HandleResolver จะมีค่า GSR เก็บไว้ที่แคช ภายในของตัวมัน แต่ถ้าไม่มีเก็บไว้ที่แคชมันก็ต้องขอ HandleResolver ตัวอื่นๆให้ช่วยติดต่อไปที่ GSH ที่ต้องการให้ โดยอาจจะผ่านโปรโตคอลที่นิยมใช้กันทุกๆไป เช่น ใช้ HTTP เพื่อรับค่า URL แล้วจึงค่อยเอนโค้ดจากค่า GSH ให้เป็น GSR เป็นต้น

7.3.4 ความสัมพันธ์ระหว่างไคลเอนต์กับ Hosting Environment

OGSI นั้นไม่ได้ทำส่วนในฝั่งที่เป็นตัวจัดเตรียมเซอร์วิส (Service Provider) แต่จะใช้โมเดลคล้ายๆกับของฝั่ง Server ที่ใช้ใน J2EE กล่าวคือ ทุกๆส่วนในมาตรฐานของกริดเซอร์วิส (เช่น ส่วนการร้องขอ, ส่วนจัดการไลฟ์ไทม์, ส่วนที่เกี่ยวกับการลงทะเบียน เป็นต้น) จะถูกซ่อนไว้ภายใน ยูสเซอร์โปรเซส เช่น การเชื่อมต่อกับไดเบอรีมาตรฐาน เป็นต้น



รูปที่ 7-5 แสดงลักษณะการทำงานของฝั่ง hosting environment

จากรูป กล่าวคือ กริดเซอร์วิสอินสแตนซ์นั้น จะฝังอยู่ในคอนเทนเนอร์ ที่ข้อความต่างๆ จะผ่านภายใน คอมโพเนนท์เหล่านี้ ในรูปแบบต่างๆ เช่น XML หรือ SOAP เป็นต้น

โดยส่วนมากแล้ว ในคอนเทนเนอร์นั้นมักจะสร้างขึ้นมาให้ใช้ในเรื่องต่างๆ เช่น จัดเตรียมการจัดการ เวลาชีวิต, หรือการตรวจสอบสิทธิและอนุญาตแบบอัตโนมัติ, การร้องขอเพื่อเข้าสู่ระบบ, การยกเลิกเซอร์วิสอินสแตนซ์เมื่อหมดไลฟ์ไทม์ และ ยกเลิกการร้องขอแบบตรงๆ

7.3.5 คุณสมบัติของกริดเซอร์วิส

จากที่กล่าวไปบทก่อนๆ แล้ว ว่าทุกอย่างที่เป็นกริดเซอร์วิสก็คือเว็บเซอร์วิสแม้ว่าลักษณะมันจะไม่เหมือนกันทุกอย่างก็ตาม ในส่วนนี้จะระบุคุณสมบัติของกริดเซอร์วิสไว้

- ใช้ส่วนของ WSDL ตามที่เว็บเซอร์วิสใช้ ตั้งแต่ WSDL 1.2 ขึ้นไป
- กำหนด เซอร์วิสเดต้า (service data) ซึ่งเป็นตัวมาตรฐานที่มาจากแสดงหรือซักถามจาก เซอร์วิสอินสแตนซ์
- แนะนำเกี่ยวกับแกนในคุณสมบัติของกริดเซอร์วิส ที่ต้องมี
 - กำหนดกริดเซอร์วิสเดสคริปชัน(Grid service description) และกริดเซอร์วิสอินสแตนซ์จากลักษณะการใช้งานจริงๆ
 - กำหนดว่า OGSi มีรูปแบบเกี่ยวกับเวลาอย่างไร
 - กำหนดการสร้าง GSH และ GSR ซึ่งใช้ในการอ้างกริดเซอร์วิสอินสแตนซ์
 - กำหนดวัฏจักรชีวิตของกริดเซอร์วิสอินสแตนซ์

7.4 เซอร์วิสเดต้า (Service Data)

การที่กริดเข้าไปใกล้การเป็นเว็บเซอร์วิสที่ทำงานเป็นสเตท ทำให้มันจำเป็นต้องมีกลไกที่จะใช้อธิบายข้อมูลในสเตทของเซอร์วิสอินสแตนซ์ที่ผู้ร้องขอคิวรี, อัปเดต, เปลี่ยนการประกาศ อย่างที่เว็บเซอร์วิสทำ ตั้งแต่ที่ใช้เว็บเซอร์วิสรวมถึงที่ใช้ภายนอกซึ่งใช้บ่งบอกข้อมูลสเตทที่เรียกว่า เซอร์วิสเคด้า

ในการเตรียมส่วนที่ใช้อธิบายรายละเอียดของเว็บเซอร์วิสที่ทำงานเป็นสเตท (เช่น กริดเซอร์วิส) มันจำเป็นที่จะต้องอธิบายสเตทที่ใช้กับส่วนภายนอกอย่างชัดเจนซึ่งก็คือสเตทที่เซอร์วิสอินสแตนซ์ถูกแสดงเมื่อผู้ใช้สร้างเซอร์วิสขึ้นมานั่นเอง สิ่งที่เป็นสำหรับการประกาศเซอร์วิสเคด้าก็คือแนวความคิดที่จะประกาศคุณสมบัติในตัวมัน ซึ่งอธิบายได้จาก Interface Definition Language (IDL) โดยที่เซอร์วิสเคด้ามันจะสามารถใช้ได้จากการอ่าน, อัปเดต หรือ ลงท้ายในข้อมูล

ซึ่ง WSDL นั้นได้กำหนด กระบวนการและข้อความสำหรับ portTypes ที่สเตทที่ถูกประกาศนั้นต้องมีการเข้าถึงได้จากภายนอกผ่านกระบวนการของเซอร์วิสที่ใช้กำหนดเซอร์วิสอินเตอร์เฟส

เท่านั้น เพื่อหลีกเลี่ยงความต้องการที่จะกำหนด กระบวนการพิเศษของเซอร์วิสเคด้าสำหรับแต่ละเซอร์วิสเคด้า โดยที่กริดเซอร์วิสนั้น จะมี portType ที่ใช้จัดกระบวนการที่ใช้ทำเซอร์วิสเคด้าด้วยตัวเอง

ตัวอย่าง มีอินเตอร์เฟสที่มีกระบวนการ op1, op2, และ op3 อยู่ โดยสมมติให้อินเตอร์เฟสนี้มีข้อมูลที่ให้สาธารณะเข้าถึงได้คือ de1, de2, de3 ซึ่งผู้ใช้ได้กำหนดรายละเอียดของอินเตอร์เฟสนี้ด้วย WSDL ซึ่ง serviceData ใน OGSi นี้ จะสร้างสืบเนื่องมาจาก WSDL ซึ่งผู้ออกแบบจะสามารถเพิ่มรายละเอียดอินเตอร์เฟสนี้ได้โดยกำหนดส่วนที่เกี่ยวกับการยอมให้เข้าถึงแบบสาธารณะของแต่ละสเตทสำหรับ de1, de2, de3 โดยการประกาศนี้จะทำให้สะดวกต่อการใช้งานของกระบวนการต่างๆบนเซอร์วิสเคด้าของเซอร์วิสอินสแตนซ์แบบมีสเตทซึ่งใช้พัฒนาอินเตอร์เฟสนี้ขึ้นมา

7.4.1 การเปรียบเทียบกับ Java Bean

ในคุณสมบัติของ serviceData ที่ OGSi ได้กำหนดสเป็คให้เป็นส่วนที่เข้าถึงข้อมูลสเตทของเว็บเซอร์วิส โดยแนวคิดของ serviceData นั้น คล้ายคลึงกับของ public instance variable หรือ field ใน object-oriented programming language เช่น Java, Smalltalk หรือ C++

ServiceData นั้นคล้ายคลึงกับคุณสมบัติของ JavaBean ซึ่งโมเดลของ JavaBean นั้นได้กำหนดรูปแบบสำหรับการทำ method signatures (getXXX/setXXX) เพื่อเข้าถึงคุณสมบัติ และผู้ช่วยในการสร้างคลาส (BeanInfo) เพื่อทำคุณสมบัติของเอกสาร โดย OGSi ได้ใช้ส่วนของ serviceData และ XML สำหรับการทำให้สิ่งที่คล้ายกันกับของ JavaBean

โดยในสเป็คของ OGSi นั้นไม่ได้เลือกใช้วิธี getXXX และ setXXX เพื่อใช้เป็นกระบวนการของ WSDL ที่นำมาใช้สำหรับแต่ละ serviceData แม้ว่าส่วนที่ใช้พัฒนาเซอร์วิสจะเลือกวิธี get และ set ด้วยตัวมันเองก็ตาม โดย OGSi ได้เลือกใช้กระบวนการ query แทนการ get และใช้วิธีการ update แทนการ set และได้ใช้ subscribe to notification สำหรับแก้ไขค่าใน serviceData ซึ่งสิ่งที่ OGSi ต้องการนั้นอย่างน้อยต้องรองรับวิธีการเหล่านี้ ซึ่งเป็นสิ่งที่อนุญาตให้เข้าถึง serviceData ในชื่อของมันของแต่ละเซอร์วิสอินสแตนซ์ อย่างไรก็ตามกระบวนการบางส่วนของ OGSi ก็นำบางส่วนมาจากอินเตอร์เฟส

อื่นๆ เพื่อรองรับการคิวรี, อัปเดต และ ลงท้าย แบบที่ยู่ยกขึ้น เช่นการ คิวรีหลายๆ serviceData ใน 1 เซอร์วิสอินสแตนซ์

โดยค่า serviceDataName ใน GridService portType นั้น ได้กำหนดไว้ตามรูปแบบเดียวกับของ BeanInfo ใน JavaBeans อย่างไรก็ตาม OGSi ได้เลือก XML (WSDL) สำหรับจัดเตรียมข้อมูลข่าวสารเกี่ยวกับ serviceData แทนที่การใช้โมเดลของ BeanInfo แบบตรงๆ

7.4.2 การแทนค่า portType ด้วย serviceData

ServiceData ที่ใช้กำหนดค่าของ portType นั้นจะเรียกว่า ServiceData Elements (SDEs) ซึ่งใน SDE จะใช้กำหนดการอ้างอิงจาก ServiceData Declarations (SDDs) โดยค่าเริ่มต้นของ SDE จะกำหนดได้จากค่า staticServiceDataValues ภายใน portType โดยค่าใดๆของ SDE จะถูกกำหนดใน portType หรืออาจจะถูกใส่ค่าจากเว็บเซอร์วิสอินสแตนซ์

ส่วนนี้เป็นตัวอย่างการใช้ gwsdl:portType ซึ่งยอมให้มีค่าปรากฏจาก namespaces อื่นๆ

```
<gwsdl:portType name="NCName"> *
  <wsdl:documentation ... /> ?
  <wsdl:operation name="NCName"> ... </wsdl:operation> ?
...
  <sd:serviceData name="NCName" ... /> *
  <sd:staticServiceDataValues?
    <some element>*
  </sd:staticServiceDataValues>
...
</gwsdl:portType>
```

สำหรับตัวอย่างนี้ จะเป็นการประกาศ 2 SDEs ของ portType ด้วยชื่อ “tns:sdl” and “tns:sdl2” ซึ่งไม่ว่าเซอร์วิสอินสแตนซ์ใดๆที่สร้างจาก portType นี้จะต้องมีค่า 2 ค่านี้

```
<wsdl:definitions xmlns:tns="xxx" targetNamespace="xxx">
  <gwsdl:portType name="exampleSDUse"> *
    <wsdl:operation name="..." ... </wsdl:operation>
...
    <sd:serviceData name="sdl" type="xsd:String"
      mutability="static"/>
    <sd:serviceData name="sdl2" type="tns:SomeComplexType"/>
...
    <sd:staticServiceDataValues>
      <tns:sdl>initValue</tns:sdl>
    </sd:staticServiceDataValues>
  </gwsdl:portType>
...
</wsdl:definitions>
```

7.4.2.1 โครงสร้างการประกาศ serviceData

จากที่ sd:serviceData ใช้กำหนดค่าใน XMML ซึ่งปรากฏใน sd:serviceDataValue และ sd:serviceData ที่ได้ออกแบบไว้หลังจากกำหนด xsd:element ของ XML ซึ่งค่าใน sd:serviceData จะใช้ 5 แอททริบิวต์เหมือนกันอย่างที่แสดงใน xsd:element: ประกอบด้วย name, type, minOccurs, maxOccurs, และ nillable ซึ่ง xsd:element อื่นๆจะห้ามใช้ภายใน sd:serviceData ซึ่งค่า sd:serviceData จะอนุญาตให้ 2 ค่าพิเศษนี้เพิ่มเติมเข้าไป ก็คือค่า mutability และค่า modifiable

โครง XML ที่ใช้กำหนด sd:serviceData นั้น ดูได้ตามนี้

```

...
    targetNamespace =
        "http://www.gridforum.org/namespaces/2003/serviceData"
    xmlns:sd =
        "http://www.gridforum.org/namespaces/2003/03/serviceData"
...

<attributeGroup name="occurs">
    <attribute name="minOccurs"
        type="nonNegativeInteger"
        use="optional"
        default="1"/>
    <attribute name="maxOccurs">
        <simpleType>
            <union memberTypes="nonNegativeInteger">
                <simpleType>
                    <restriction base="NMTOKEN">
                        <enumeration value="unbounded"/>
                    </restriction>
                </simpleType>
            </union>
        </simpleType>
    </attribute>
</attributeGroup>

<complexType name="ServiceDataType">
    <sequence>
        <any namespace="##any" minOccurs="0" maxOccurs="unbounded"/>
    </sequence>
    <attribute name="name" type="NCName"/>
    <attribute name="type" type="QName"/>
    <attribute name="nillable"
        type="boolean"
        use="optional"
        default="false"/>
    <attributeGroup ref="sd:occurs"/>
    <attribute name="mutability" use="optional" default="extendable">
        <simpleType>
            <restriction base="string">
                <enumeration value="static"/>
                <enumeration value="constant"/>
                <enumeration value="extendable"/>
                <enumeration value="mutable"/>
            </restriction>
        </simpleType>
    </attribute>
    <attribute name="modifiable" type="boolean" default="false"/>
    <anyAttribute namespace="##other" processContents="lax"/>
</complexType>

```



```

<element name="serviceData" type="sd:ServiceDataType"/>

<xsd:complexType name="ServiceDataValuesType">
  <xsd:sequence>
    <xsd:any namespace="##any" minOccurs="0"
      maxOccurs="unbounded" />
  </xsd:sequence>
</xsd:complexType>

<element name="serviceDataValues"
  type="sd:ServiceDataValuesType" />

<element name="staticServiceDataValues"
  type="sd:ServiceDataValuesType" />

```

โดยค่าของ แอททริบิวต์แต่ละส่วน serviceData คือตามนี้

- maxOccurs = (nonNegativeInteger | unbounded) : default to 1
 - ค่านี้แสดงค่าตัวเลขที่มากที่สุดของ serviceData ที่สามารถปรากฏได้ในค่า serviceDataValues ของเซอวีวีส์อินสแตนซ์ หรือ ค่า staticServiceDataValues ของ portType
- minOccurs = nonNegativeInteger : default to 1
 - ค่านี้แสดงค่าตัวเลขที่น้อยที่สุดของ serviceData ที่สามารถปรากฏได้ในค่า serviceDataValues ของเซอวีวีส์อินสแตนซ์ หรือ ค่า staticServiceDataValues ของ portType
 - ถ้าค่านี้เป็น 0 แสดงว่าค่าของ serviceData นี้เป็นแบบ optional
- name = NCName and {target namespace}
 - ชื่อของ serviceData ต้องเป็นเอกเทศกันระหว่าง sd:serviceData และ xsd:element ทั้งหมดที่ใช้ประกาศใน namespace เป้าหมายของ wsdl:definition element
 - การรวมกันของชื่อใน serviceData และ targetNamespace ของ wsdl:element จาก QName นั้นจะยอมให้มีการอ้างเอกเทศของค่า serviceData นี้
- nillable = boolean : default to false
 - ค่านี้จะแสดงว่า serviceData มีค่าที่เป็น nil (ซึ่งเป็นค่าที่มี xsi:nil เป็นค่า true) ตัวอย่าง


```

<serviceDataElement name="foo" type="xsd:string"
  nillable="true" />

<foo xsi:nil="true"/>

```
- type = QName
 - ค่านี้ใช้กำหนดชนิดของ XML ของ serviceData

- modifiable = “boolean” : default to false
 - ถ้าค่านี้เป็นจริง จะหมายความว่าเป็นผู้ร้องขอโดยตรงเพื่อขออัปเดตค่า serviceData ผ่านคำสั่ง serServiceData ซึ่งเป็นการตั้งแบบฟันค่า cardinality (minOccurs, maxOccurs) และ mutability ถ้าค่านี้เป็นเท็จ จะหมายถึงค่าของ serviceData ควรจะถูกพิจารณาแบบ read only เท่านั้น โดยผู้ร้องขอ แม้ว่าค่านั้นจะเปลี่ยนผลลัพธ์ของกระบวนการอื่นๆบนอินเทอร์เน็ตเฟสนี้ก็ตาม
- mutability = “static” | “constant” | “extendable” | “mutable” : default to extendable
 - ค่านี้แสดงว่า serviceData จะสามารถเปลี่ยนได้อย่างไร

7.4.2.2 การใช้ serviceData โดยใช้ตัวอย่างจาก GridService portType

เพื่อแสดงว่า serviceData ถูกใช้อย่างไร จะอธิบายได้โดยโค้ดส่วนนี้

```
<wsdl:definitions ...
<gwsdl:portType name="GridService" ...>
  <wsdl:operation name=...> ... </wsdl:operation>
  ...

  <sd:serviceData name="interface" type="xsd:QName"
    minOccurs="1" maxOccurs="unbounded"
    mutability="constant"/>
  <sd:serviceData name="serviceName" type="xsd:QName"
    minOccurs="0" maxOccurs="unbounded"
    mutability="mutable" nillable="false"/>
  <sd:serviceData name="factoryHandle"
    type="ogsi:HandleType"
    minOccurs="1" maxOccurs="1"
    mutability="constant" nillable="true"/>
  <sd:serviceData name="gridServiceHandle"
    type="ogsi:HandleType"
    minOccurs="0" maxOccurs="unbounded"
    mutability="extendable"/>
  <sd:serviceData name="gridServiceReference"
    type="ogsi:ReferenceType"
    minOccurs="0" maxOccurs="unbounded"
    mutability="mutable"/>
  <sd:serviceData name="findServiceDataExtensibility"
    type="ogsi:OperationExtensibilityType"
    minOccurs="1" maxOccurs="unbounded"
    mutability="static"/>
  <sd:serviceData name="terminationTime" type="ogsi:terminationTime"
    minOccurs="1" maxOccurs="1"
    mutability="mutable"/>
  <sd:serviceData name="setServiceDataExtensibility"
    type="ogsi:OperationExtensibilityType"
    minOccurs="2" maxOccurs="unbounded"
    mutability="static" />
  ...
</gwsdl:portType>
```

ส่วนนี้คือตัวอย่างการเซตค่า serviceData สำหรับกริดเซอร์วิสอินสแตนซ์

```

...
  xmlns:crm="http://gridforum.org/namespaces/2002/11/crm"
  xmlns:tns="http://example.com/exampleNS"
  xmlns="http://example.com/exampleNS">

<sd:serviceDataValues>
  <ogsi:interface>crm:GenericOSPT</ogsi:interface>
  <ogsi:interface>ogsi:GridService</ogsi:interface>

  <ogsi:serviceName>ogsi:interface
  </ogsi:serviceName>
  <ogsi:serviceName>ogsi:serviceName
  </ogsi:serviceName>
  <ogsi:serviceName>ogsi:factoryHandle
  </ogsi:serviceName>
  <ogsi:serviceName>ogsi:gridServiceHandle
  </ogsi:serviceName>
  <ogsi:serviceName>ogsi:gridServiceReference
  </ogsi:serviceName>
  <ogsi:serviceName>ogsi:findServiceDataExtensibility
  </ogsi:serviceName>
  <ogsi:serviceName>ogsi:terminationTime
  </ogsi:serviceName>
  <ogsi:serviceName>ogsi:setServiceDataExtensibility
  </ogsi:serviceName>

  <ogsi:factoryHandle>someURI</ogsi:factoryHandle>

  <ogsi:gridServiceHandle>someURI</ogsi:gridServiceHandle>
  <ogsi:gridServiceHandle>someOtherURI</ogsi:gridServiceHandle>

  <ogsi:gridServiceReference>...</ogsi:gridServiceReference>
  <ogsi:gridServiceReference>...</ogsi:gridServiceReference>

  <ogsi:findServiceDataExtensibility
    inputElement="ogsi:queryByServiceDataNames" />

  <ogsi:terminationTime after="2002-11-01T11:22:33"
    before="2002-12-09T11:22:33"/>

  <ogsi:setServiceDataExtensibility
    inputElement="ogsi:setByServiceDataNames" />
  <ogsi:setServiceDataExtensibility
    inputElement="ogsi:deleteByServiceDataNames" />

</sd:serviceDataValues>

```

7.4.3 Mutability

ค่า mutability บน serviceData นั้นจะเป็นส่วนที่แสดงว่าค่าของ serviceData นั้นจะเปลี่ยนได้
อย่างไรในโลกฟิสิกส์

mutability = "static" แสดงว่า ค่า SDE ถูกแทนให้ด้วยการประกาศของ WSDL
(staticServiceDataValue) และต้องคงมีค่า instance อื่นๆของ portType นั้น โดยค่า "static" นั้น
เปรียบเสมือนค่าตัวแปรที่เป็นสมาชิกของคลาสในภาษาโปรแกรมมิ่ง

mutability = “constant” แสดงว่า SDE ถูกแทนให้โดยการสร้างกริดเซอร์วิสอินสแตนซ์และต้องไม่มีการเปลี่ยนแปลงระหว่างช่วงที่อยู่ในเวลาชีวิตของกริดเซอร์วิสอินสแตนซ์นั้นจนกระทั่งมีการเซ็ค่า

mutability = “extendable” แสดงว่าอย่างน้อย 1 ค่าของ SDE นั้นเป็นส่วนหนึ่งของไลฟ์ไทม์ของ กริดเซอร์วิสอินสแตนซ์ ซึ่งมีค่าใหม่เพิ่มขึ้นมาได้ แต่ค่าเหล่านั้นต้องไม่ถูกลบออกไป

mutability = “mutable” แสดงว่าค่าใดๆใน SDE อาจถูกลบไปบางช่วงเวลาและบางที่อาจถูกเพิ่มขึ้นมา

7.4.4 serviceDataValues

แต่ละเซอร์วิสอินสแตนซ์จะเกี่ยวข้องกับกลุ่มของ serviceData ซึ่งค่าใน serviceData จะถูกกำหนดโดยหลายๆ portType จากอินเทอร์เฟซของเซอร์วิสนั้น และ บางที่อาจจะมีการเพิ่มค่าระหว่างช่วงเวลาที่รันอยู่ ซึ่งเรียกกลุ่มของ serviceData ที่เกี่ยวข้องกับเซอร์วิสอินสแตนซ์นี้ว่า “serviceData set” ซึ่งบางที จะอ้างอิงกับกลุ่มของ serviceData ที่ ไปรวมกับค่าของ serviceData ที่ถูกประกาศไว้ด้วยอินเทอร์เฟซของ portType

แต่ละเซอร์วิสอินสแตนซ์นั้นต้องมีเอกสาร XML ที่เป็นแบบลอจิก พร้อมด้วยค่า serviceDataValues ที่บรรจุ serviceData ไว้ ซึ่งการเขียนเซอร์วิสนั้นจะอิสระที่จะเลือกใช้ว่าค่า SDE นั้นถูกเก็บไว้แบบไหน เช่น บางที่อาจจะไม่ได้เก็บไว้ในรูป XML แต่เก็บเป็นตัวแปรอินสแตนซ์ซึ่งอาจเปลี่ยนรูปจาก XML หรือ วิธีการเอนโค้ดแบบอื่นๆเท่าที่จำเป็น

7.4.5 การกำหนดค่าเริ่มต้นของ SDE

ในการใช้ค่าของ staticServiceDataValues นั้น portType จะต้องมีการประกาศค่าเริ่มต้นสำหรับ serviceData ใดๆก็ตามด้วย mutability ที่เซ็ไว้แบบ “static” เท่านั้น แม้ว่า serviceData จะถูกกำหนดไว้แบบ local หรือต่อเนื่องมาจาก portTypes ค่าเริ่มต้นนั้นจะต้องกำหนดในหลายๆ portTypes ข้างในอินเทอร์เฟซนั้นๆ トラバที่ค่าเริ่มต้นนั้นยังไม่เกินค่า maxOccurs ที่ระบุไว้ ตัวอย่างนี้จะเป็นการกำหนดค่าเริ่มต้น 2 ค่าสำหรับ tns:otherSD ให้เป็น “1” และ “2”

```

<wsdl:definitions xmlns:tns="xxx" targetNamespace="xxx">
  <gwsdl:portType name="otherPT">
    <wsdl:operation name=...> ... </wsdl:operation>
  ...
    <sd:serviceData name="otherSD" type="xsd:String"
      mutability="static" maxOccurs="unbounded"/>

    <sd:staticServiceDataValues>
      <tns:otherSD>initial value 1</tns:otherSD>
    </sd:staticServiceDataValues>
  ...
</gwsdl:portType>

  <gwsdl:portType name="exampleSDUse" extends="tns:otherPT">
    <wsdl:operation name=...> ... </wsdl:operation>
  ...
    <sd:serviceData name="sd1" type="xsd:String"
      mutability="static" />
    <sd:serviceData name="sd2" type="tns:SomeComplexType"/>

    <sd:staticServiceDataValues>
      <tns:sd1>an initial value</tns:sd1>
      <tns:otherSD>initial value 2</tns:otherSD>
    </sd:staticServiceDataValues>
  ...
</gwsdl:portType>
...
</wsdl:definitions>

```

7.4.6 การรวมกันของ SDE ภายในโครงสร้างของ portType

WSDL 1.2 นั้นจะแนะนำเกี่ยวกับหลายๆ portType และการออกแบบภายในขอบเขตของ gwsdl ซึ่ง portType จะสามารถสืบมาจาก 0 หรือมากกว่านี้จาก portTypes อื่นๆ โดยที่ไม่ใช่ความสัมพันธ์ตรงๆระหว่าง wsdl:Service และ portType ที่ถูกรับรองโดยโมเดลของเซอร์วิสในรูปแบบของ WSDL ซึ่งกลุ่มของ portType นั้นจะสร้างโดยเซอร์วิสที่สืบทอดมาจากค่าลูกของเซอร์วิสและค่าที่อ้างจากค่า port โดยค่า กลุ่มของ portTypes และ portTypes จะถูกกำหนดด้วยเซอร์วิสอินเตอร์เฟซที่สมบูรณ์

serviceData ที่ถูกกำหนดโดยเซอร์วิสนั้นเป็นเซตที่รวมกันของ serviceData ที่รวมกันของแต่ละ portType ในอินเตอร์เฟซที่สมบูรณ์ที่สร้างมาจากเซอร์วิสอินสแตนซ์ เพราะว่าค่า serviceData จะเป็นเอกเทศจาก QName ซึ่งเป็นกลุ่มจะปรากฏขึ้นเพียงครั้งเดียวในเซตของ serviceData ยกตัวอย่างเช่น ถ้า portType มีชื่อ "pt1" และ "pt2" ซึ่งทั้งคู่กำหนดโดย serviceData ที่ชื่อ "tns:sd1" และมี portType ที่ชื่อ "pt3" ซึ่งสืบทอดมาจาก "pt1" และ "pt2" โดยที่มีแค่ 1 serviceData เท่านั้นที่ชื่อ "tns:sd1"

พิจารณาได้จากตัวอย่างนี้

```

<gwsdl:portType name="pt1">
  <sd:serviceData name="sd1" ... />
</gwsdl:portType>

<gwsdl:portType name="pt2" extends="pt1">
  <sd:serviceData name="sd2" ... />
</gwsdl:portType>

<gwsdl:portType name="pt3" extends="pt1">
  <sd:serviceData name="sd3" ... />
</gwsdl:portType>

<gwsdl:portType name="pt4" extends="pt2 pt3">
  <sd:serviceData name="sd4" ... />
</gwsdl:portType>

```

ซึ่งกลุ่มของ serviceData ได้กำหนด 4 portTypes ดังที่แสดงไว้ดังตารางข้างล่าง

if a service implements...	its serviceData set contains...
Pt1	sd1
Pt2	sd1, sd2
Pt3	sd1, sd3
Pt4	sd1, sd2, sd3, sd4

ตารางที่ 7-2 แสดงกลุ่มของ serviceData

7.4.6.1 ค่าเริ่มต้นของ SDE แบบ static ภายในอินเทอร์เฟซของ portType

ค่าเริ่มต้นของ SDE จะสามารถรวมกันลงไปที่อินเทอร์เฟซของ portType ได้ อย่างไรก็ตาม ค่า cardinality ที่ต้องการ (minOccurs และ maxOccurs) จะต้องถูกกันไว้ เช่น ในตัวอย่าง เซอร์วิสอินสแตนซ์ที่สร้าง pt1 จะมีค่า <sd1> 1 </sd1> สำหรับ SDE ที่ชื่อ sd1

```

<gwsdl:portType name="pt1">
  <sd:serviceData name="sd1" minOccurs="1" maxOccurs="1"
    mutability="static"/>
  <sd:staticServiceDataValues>
    <sd1>1</sd1>
  </sd:staticServiceDataValues>
</gwsdl:portType>

```

ต่อจากนี้จะเป็นการสร้าง pt2 โดยสืบทอดค่า <sd1> 1 </sd1> สำหรับ SDE ที่ชื่อ sd1 และจะมีค่า <sd2> 2 </sd2> สำหรับ SDE ที่ชื่อ sd2

```

<gwsdl:portType name="pt2" extends="pt1">
  <sd:serviceData name="sd2" minOccurs="1" maxOccurs="1"
    mutability="static"/>
  <sd:staticServiceDataValues>
    <sd2>2</sd2>
  </sd:staticServiceDataValues>
</gwsdl:portType>

```

ค่าเซอร์วิสอินสแตนซ์ที่สร้าง pt3 ก็จะมี 2 ค่าคือ <sd3> 3a </sd3> และ <sd3>3b</sd3> สำหรับ SDE ที่ชื่อ sd3 ซึ่งควรจะสืบทอดค่าจาก SDE ที่ชื่อ sd1 ด้วย

```
<gwsdl:portType name="pt3" extends="pt1">
  <sd:serviceData name="sd3" minOccurs="1" maxOccurs="unbounded"
    mutability="static"/>
  <sd:staticServiceDataValues>
    <sd3>3a</sd3>
    <sd3>3b</sd3>
  </sd:staticServiceDataValues>
</gwsdl:portType>
```

ค่าเซอร์วิสอินสแตนซ์ที่สร้าง pt4 จะสืบทอดจากค่า sd1 ที่กำหนดโดย pt1 แต่การขาดไปของค่า staticServiceDataValues ที่แสดงให้เห็นโดยไม่มีค่าเริ่มต้นสำหรับ sd4 (แม้ว่ากำหนดจาก portType ที่สืบทอดมาจาก pt4)

```
<gwsdl:portType name="pt4" extends="pt1">
  <sd:serviceData name="sd4" minOccurs="0" maxOccurs="unbounded"
    mutability="static"/>
</gwsdl:portType>
```

เซอร์วิสอินสแตนซ์ที่สร้าง pt5 จะไม่สามารถสร้างได้ ตั้งแต่ที่ไม่มีค่าเริ่มต้นสำหรับ sd5 และตั้งแต่ที่ค่า minOccurs มีค่ามากกว่า 0 โดยจะมีความผิดพลาดเกิดขึ้นและแสดงออกผ่านการสร้างอินสแตนซ์ โดย portType ชนิดนี้จะพบผ่านการกำหนด “abstract” portType ภายใน portTypes ที่มาจากการกำหนดค่าสำหรับ SDEs ด้วย minOccurs ที่มากกว่า 0

```
<gwsdl:portType name="pt5" extends="pt1">
  <sd:serviceData name="sd5" minOccurs="1" maxOccurs="unbounded"
    mutability="static"/>
</gwsdl:portType>
```

เซอร์วิสอินสแตนซ์ที่สร้าง pt6 ก็ไม่สามารถสร้างได้ ตั้งแต่ที่ portType กำหนดค่าเพิ่มเติมของ SDE ที่ชื่อ sd1 (เรียกใช้ค่าที่สืบทอดมาจาก pt1) ซึ่งมีค่ามากกว่า maxOccurs ของ SDE ที่ชื่อ sd1 โดยจะมีความผิดพลาดแจ้งให้ทราบผ่านการสร้างอินสแตนซ์เช่นกัน

```
<gwsdl:portType name="pt6" extends="pt1">
  </sd:staticServiceDataValues>
  <sd1>6</sd1>
  <sd:staticServiceDataValues>
</gwsdl:portType>
```

เซอร์วิสอินสแตนซ์ที่สร้าง pt7 จะมีเซตที่น่าสนใจของ serviceData อยู่ อย่างแรกคือ เป็นค่าเดี่ยว <sd1>1</sd1> สำหรับ SDE ชื่อ sd1 แม้ว่าจะมีการสืบทอด pt1 ผ่าน pt2 และ pt3 ซึ่งค่าเริ่มต้นของ sd1

จะไม่ซ้ำกัน โดยค่า <sd2>2</sd2> เป็นค่าเดียวสำหรับ SDE ที่ชื่อ sd2 ซึ่งสืบทอดมาจาก pt2 และ SDE ที่ชื่อ pt3 จะมีค่า 3 ค่า คือ <sd3>3a</sd3>, <sd3>3b</sd3> (สืบทอดมาจาก pt3) และ <sd3>7</sd3> ซึ่งกำหนดแบบ local นอกจากนี้ยังมีค่าแบบ local ที่กำหนดโดย SDE ที่ชื่อ sd7 คือ <sd7>7</sd7>

```
<gwsdl:portType name="pt7" extends="pt2 pt3">
  <sd:serviceData name="sd7" minOccurs="1" maxOccurs="1"
    mutability="static"/>
  <sd:staticServiceDataValues>
    <sd7>7</sd7>
    <sd3>7</sd3>
  </sd:staticServiceDataValues>
</gwsdl:portType>
```

โดยสรุปแล้ว ค่า SDE จะรวมอินเตอร์เฟซของ portType ถ้าผลของ SDE ไปกระทบกับ cardinality (ค่าที่น้อยกว่า minOccurs หรือ มากกว่า maxOccurs) ของ SDE แล้ว จะมีการแจ้งความผิดพลาดขึ้นเมื่อเว็บเซอร์วิสนั้นได้ถูกสร้างขึ้น

7.4.7 ค่า serviceData แบบ dynamic

แม้ว่าหลายๆ serviceData จะถูกกำหนดโดยรูปแบบของเซอร์วิสแล้ว บางสถานการณ์ก็อาจจะใช้การเพิ่มหรือการลบ serviceData แบบ dynamic หรือ จากอินสแตนซ์มาช่วย นี่หมายความว่าบางการอัปเดตจะเป็นการสร้างแบบเจาะจง ตัวอย่างเช่น เซอร์วิสอินสแตนซ์จะมีกระบวนการสร้างค่าเซอร์วิสเดต้าใหม่ๆเพิ่มขึ้นมาได้

โดย GridService portType ที่ใช้กับ SDEs แบบ dynamic นั้น จะประกอบไปด้วยค่า serviceData ที่ชื่อ "serviceName" ที่ใช้แสดง serviceData ที่กำหนดในปัจจุบัน คุณสมบัติของอินสแตนซ์นี้จะตอบค่าซูปเปอร์เซตของเซอร์วิสเดต้าที่ประกาศโดย GWSDL กลับไป โดยกำหนดรูปแบบของเซอร์วิส ที่ยอมให้ผู้ร้องขอ ใช้ กระบวนการ subscribe ที่ใช้เปลี่ยน serviceDataSet และ ใช้ กระบวนการ findServiceData ที่ใช้เลือกค่าปัจจุบันของ serviceDataSet

7.5 คุณสมบัติสำหรับแก่นของกริดเซอร์วิส

จะแบ่งออกเป็นกลุ่มๆของทุกๆกริดเซอร์วิส ดังนี้

1. Service Description และ Service Instance
2. รูปแบบของเวลาที่ใช้ใน OGSI
3. ค่าไลฟ์ไทม์ของ XML
4. รูปแบบการให้ชื่อและการจัดการการเปลี่ยนแปลง
5. การให้ชื่อกริดเซอร์วิสอินสแตนซ์
6. วัฏจักรชีวิตของกริดเซอร์วิส
7. การจัดการเมื่อเกิดกระบวนการผิดพลาด

8. กระบวนการที่ใช้สืบทอด

7.5.1 Service Description และ Service Instance

ใน OGSi นั้นได้แบ่งแยกระหว่าง description กับ instance ของกริดเซอร์วิส ดังนี้

- Grid Service Description จะเป็นส่วนที่แจ้งบอกว่า client จะตอบโต้กับเซอร์วิสอินสแตนซ์ได้อย่างไร โดยที่ description นี้จะเป็นอิสระต่ออินสแตนซ์อื่นๆ ภายในเอกสารของ WSDL นั้น Grid Service Description จะมาจากการสืบทอดจาก portType (เช่น portType ที่อ้างอิงจากคำพอร์ทลูกของ wsdl:service) ของอินสแตนซ์ พร้อมด้วย portType ที่เกี่ยวข้อง (รวมถึงการประกาศ SDE) , การผูก, ข้อความ และ การกำหนดชนิดต่างๆ

- Grid Service Description ที่ถูกใช้ในเวลาเดียวกันโดย Grid Service Instance นั้น จะมีลักษณะดังต่อไปนี้

1. เกิดเป็นรูปร่างจากที่มาจากการ description ของเซอร์วิสที่แจ้งว่าจะตอบโต้ได้อย่างไร
2. มี Grid Service Handles อยู่ 1 คำหรือ มากกว่านั้น
3. มี Grid Service Reference อยู่ 1 คำหรือ มากกว่านั้นที่ใช้อ้างอิง

โดย service description จะใช้ในเบื้องต้นอยู่ 2 อย่าง คือ ใช้เพื่ออธิบายเกี่ยวกับเซอร์วิสซึ่งส่วนนี้จะสามารถใช้เครื่องมือช่วยในการสร้างขึ้นมาได้อย่างอัตโนมัติ และใช้เพื่อค้นหาเซอร์วิส เช่น เพื่อหาเซอร์วิสอินสแตนซ์ที่สร้างขึ้นมาโดยเฉพาะจาก service description หรือหา factory ที่สามารถสร้างอินสแตนซ์โดยเฉพาะจาก service description

โดยการบ่งบอกถึง service description นั้น ต้องมี 2 ส่วนคือ syntax และ semantics โดยที่ syntax จะอธิบายได้ด้วย WSDL portType และ semantics จะใช้แสดงผ่านชื่อที่ใช้ใน portType เช่น เมื่อกำหนดกริดเซอร์วิสขึ้นมา โดยใช้ชื่อที่เป็นเอกเทศขึ้นมาหลายๆชื่อ semantic จะพยายามที่จะเกี่ยวข้องกับแต่ละ ชื่อนั้นๆ โดยชื่อเหล่านี้จะสามารถใช้โดย client เพื่อค้นหาเซอร์วิสโดยความต้องการของ semantics โดยจะค้นหาเซอร์วิสอินสแตนซ์และแพลตฟอร์มด้วยชื่อพิเศษนั้นๆ การใช้ขอบเขตของชื่อเหล่านี้จะเป็นการเตรียมสำหรับการใช้ชื่อที่จะเป็นเอกเทศต่อกัน

7.5.2 รูปแบบของเวลาที่ใช้ใน OGSi

ส่วนที่สำคัญส่วนหนึ่งที่เกิดขึ้นกับการใช้งานของหลายๆกลุ่มในการกระจายของกริดนั้น ก็คือการแสดงเวลานั่นเอง ยกตัวอย่างเช่น มีข้อมูลที่ต้องส่งด้วย timestamps ตามลำดับ เพื่อให้ข้อมูลนั้นจะมีประโยชน์กับผู้ที่นำไปใช้ เพราะบางที client อาจจะจำเป็นต้องใช้เซอร์วิสอินสแตนซ์นั้นในช่วงเวลาที่สมควรใช้งานอยู่ และหลายๆเซอร์วิสก็ต้องอาศัยความเข้าใจเรื่องลำดับให้ถูกต้องเพื่อให้ client สามารถจัดการและตอบโต้กลับมันได้ทันที

เวลามาตรฐานแบบ GMT นั้น ถูกนำมาใช้ในกริดเซอร์วิส ที่จะยอมให้เวลาโดยรวมกลมกลืนกัน อย่างไรก็ตามการใช้เวลา GMT เป็นมาตรฐานนั้นจะไม่ทำให้เกิดการ synchronization กับทุกๆ

เครื่องได้ เพราะเวลาของแต่ละเครื่องย่อมไม่ตรงกันทั้งหมด ซึ่งส่วนนี้จำเป็นต้องใช้วิธีพิเศษที่จะทำให้เวลานั้น synchronize กันเพื่อคุณภาพของงาน

โดยกริดเซอร์วิสทั้งฝั่ง hosting environments และฝั่ง client จะต้องใช้เวลาร่วมกันผ่าน Network Time Protocol (NTP) หรือ อาจจะใช้ฟังก์ชันพิเศษที่จะทำให้เวลานั้นตรงตาม GMT อยู่เสมอ อย่างไรก็ตาม client และ service ต้องมีการยอมรับข้อความที่บรรจุค่าเกี่ยวกับเวลาไว้แล้วเกินขอบเขตบ้าง เพราะ การ synchronize นั้นไม่เพียงพอ ซึ่งอาจทำให้ข้อมูลที่ส่งมาไม่ถึงได้

ในบางกรณีก็อาจต้องการเวลาที่เป็น 0 หรือเป็นอนันต์ก็มี โดยเวลาที่เป็น 0 นั้นจะใช้แสดงแทนเวลาที่ผ่านไปแล้วในอดีต ส่วนเวลาที่เป็นค่าอนันต์จะใช้สำหรับแสดงความคิดเกี่ยวกับเวลา โดยในขอบเขตของชื่อใน ogsi นั้นจะแทนที่ค่าเวลาพิเศษนั้นๆ ใน xsd:dateTime ดังตัวอย่าง

```
... targetNamespace =  
    "http://www.gridforum.org/namespaces/2003/03/OGSI"  
  
<simpleType name="ExtendedDateTimeType">  
    <union memberTypes="ogsi:InfinityType xsd:dateTime"/>  
</simpleType>  
  
<simpleType name="InfinityType">  
    <restriction base="string">  
        <enumeration value="infinity"/>  
    </restriction>  
</simpleType>
```

7.5.3 ค่าไถ่ใหม่ของ XML

ตั้งแต่ที่ serviceData จะต้องแทนไดนามิกสเตทของเซอร์วิสอินสแตนซ์ จึงเป็นเรื่องจำเป็นที่จะต้องเข้าใจเกี่ยวกับไถ่ใหม่ที่ถูกต้อง โดยไคลเอนท์จะต้องใช้ข้อมูลที่เกี่ยวข้องกับเวลาสำหรับการใช้ในเรื่องต่างๆ จนกระทั่งถึงการอิสระที่จะปฏิเสธข้อมูลนั้นๆ ไปด้วย

ซึ่งจะใช้ 3 แอททริบิวต์ของ XML ที่เป็นข้อมูลที่เกี่ยวข้องกับค่าไถ่ใหม่ คือ

- ogsi:goodFrom: ใช้กำหนดเวลาจากที่ซึ่งข้อมูลถูกต้อง ส่วนนี้ใช้เวลาตอนที่เริ่มสร้างมาใช้
- ogsi:goodUntil: ใช้กำหนดเวลาจนกระทั่งข้อมูลยังถูกต้อง ถ้าใช้ส่วนนี้จะมีคุณสมบัติที่ดีกว่า

หรืออย่างน้อยเทียบเท่ากับแบบ goodForm

- ogsi:availableUntil: ใช้กำหนดเวลาจนกระทั่งถึงช่วงที่ค่าที่คาดหวังยังคงใช้ได้ บางทีอาจจะมีการอัปเดตค่านี้ด้วย ซึ่งเวลาแบบนี้ไคลเอนท์จะต้องได้รับค่าที่อัปเดตนี้ด้วย หลังจากนั้นแล้วไคลเอนท์ ไม่ควรที่จะได้รับค่านี้อีกที เนื่องจากอาจมีปัญหาเกี่ยวกับกฎหรือสิ่งที่สำคัญอื่นๆ คุณสมบัติแบบนี้ต้องดีกว่าหรือเทียบเท่ากับแบบ goodFrom

ตัวอย่างการใช้ XML สำหรับแอททริบิวต์เหล่านี้

```

targetNamespace = "http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:attribute name="goodFrom" type="ogsi:ExtendedDateTimeType"/>
<xsd:attribute name="goodUntil" type="ogsi:ExtendedDateTimeType"/>
<xsd:attribute name="availableUntil" type="ogsi:ExtendedDateTimeType"/>

<xsd:attributeGroup name="LifeTimePropertiesGroup">
  <xsd:attribute ref="ogsi:goodFrom" use="optional"/>
  <xsd:attribute ref="ogsi:goodUntil" use="optional"/>
  <xsd:attribute ref="ogsi:availableUntil" use="optional"/>
</xsd:attributeGroup>

```

ซึ่งจะใช้ serviceData ค่านี้เพื่อกำหนดไลฟ์ไทม์ตามนี้

```

<wsdl:definitions
  targetNamespace="http://example.com/ns"
  xmlns:n1="http://example.com/ns"
  ... >

  <wsdl:types>
    <xsd:schema ...
      "targetNamespace=http://example.com/ns"
      ...>
      <xsd:complexType name="MyType">
        <xsd:sequence>
          <xsd:element name="e1" type="xsd:string" minOccurs="1"/>
          <xsd:element name="e2" type="xsd:string" minOccurs="1"/>
          <xsd:element name="e3" type="xsd:string" minOccurs="0"/>
        </xsd:sequence>
        <anyAttribute namespace="##any"/>
      </xsd:complexType>
    </xsd:schema>
  </wsdl:types>
  ...
  <gwsdl:portType name="MyPortType">
    ...
    <sd:serviceData name="mySDE" type="n1:MyType"
      minOccurs="1" maxOccurs="1"
      mutability="mutable"/>
    ...
  </gwsdl:portType>
  ...
</wsdl:definitions>

```

และภายในเซอร์วิสอินสแตนซ์นั้นจะต้องมีค่า serviceDataValues ดังนี้

```

<sd:serviceDataValues
  <n1:mySDE
    goodFrom="2002-04-27T10:20:00.000-06:00"
    goodUntil="2002-04-27T11:20:00.000-06:00"
    availableUntil="2002-04-28T10:20:00.000-06:00">
    <n1:e1>
      abc
    </n1:e1>
    <n1:e2 ogssi:goodUntil="2002-04-27T10:30:00.000-06:00">
      def
    </n1:e2>
    <n1:e3 ogssi:availableUntil="2002-04-27T20:20:00.000-06:00">
      ghi
    </n1:e3>
  </n1:mySDE>
</sd:serviceDataValues>

```

โดยใช้ goodFrom และ goodUntil ของ n1:mySDE นั้นจะอ้างถึงค่าทุกทั้งหมด ที่แอททริบิวต์ เหล่านั้นกำหนดไว้ใน SDE เป็นค่าไลฟ์ไทม์ที่คาดหวังไว้สำหรับค่านั้น ซึ่งในตัวอย่างนี้จะใช้จาก 10.20am ไปจนถึง 11.20 am ในวันที่ 27 เดือนเมษายน ปี 2002 โดยผู้ที่นำไปใช้จะได้รับ SDE นี้ที่เวลา 10.20am ดังนั้นแม้ว่าผู้ที่ใช้งานจะได้รับช้ากว่านั้นอีก 1 ชม. ก็จะสามารถใช้งานได้ถูกต้อง โดยที่ไคลเอนต์บางทีอาจจะคิวรีเซอร์วิสอินสแตนซ์อีกครั้งเมื่อเวลา 11.20 เพื่อรับค่าใหม่ก็ได้

7.5.4 รูปแบบการให้ชื่อและการจัดการการเปลี่ยนแปลง

ส่วนสำคัญของระบบแบบกระจายนั้นก็ต้องยอมรับค่าที่เปลี่ยนแปลงตลอดเวลาได้ เพื่อให้ได้สิ่งนี้แล้ว client จึงต้องการที่จะเลือกเซอร์วิสที่เปลี่ยนแปลงไปได้เพื่อที่จะนำใช้ในการทำงานของเขา ส่วนนี้จะเป็นส่วนที่แสดงถึงการจัดการส่วนนี้ของ OGSI

7.5.4.1 ปัญหาของการจัดการการเปลี่ยนแปลง

ในกริดเซอร์วิสอินสแตนซ์นั้นจะมีการจำกัดความได้อยู่ 2 ลักษณะคือ

1. เป็นรูปแบบคุณสมบัติของมัน ซึ่งแบบนี้ กริดเซอร์วิสอินสแตนซ์จะกำหนดโดย service description ซึ่งประกอบไปด้วย portTypes, operations, serviceData, messages และ types
2. เป็นรูปแบบการทำงาน ส่วนนี้อาจจะแสดงมาจากแบบแรกด้วย โดยจะบอกว่าแท้จริงแล้วมันให้กริดเซอร์วิสอินสแตนซ์ได้อย่างไร ซึ่งรูปแบบการทำหรือความผิดพลาดจะแสดงผลออกมาที่เซอร์วิสอินสแตนซ์

สำหรับไคลเอนต์เพื่อที่จะค้นหาและใช้งานกริดเซอร์วิสอินสแตนซ์ได้อย่างถูกต้องนั้นไคลเอนต์ จะต้องเลือกจากคำจำกัดความของ 2 ชนิดที่กล่าวมา แต่เมื่อลองพิจารณาดูการที่ Grid service description บ่งบอกค่าออกมาเป็น portType นั้นเป็นสิ่งที่ไคลเอนต์ต้องการจริงๆ? และก็ส่วนที่เป็นโครงของการทำงานนั้นจะไม่มีผลผิดพลาดจนต้องแก้ไขเลยหรือ?

ต่อมา grid service description นั้นจำเป็นจะต้องมีออกมาอยู่ตลอดเวลา ถ้ากริดเซอร์วิสอินสแตนซ์นั้นสืบทอดมาจากส่วนข้างหลัง แล้วไคลเอนต์ต้องการส่วนนั้นก็ควรต้องรองรับส่วนที่มันเป็น

ส่วนที่สืบทอดมาได้ด้วย เช่น เพิ่ม operation หรือ service description ใหม่ขึ้นมา ซึ่งบางทีอาจเกิดจากการเปลี่ยนแปลงที่จำเป็น เช่น แก้ไขบั๊ก, ทำให้แก้ไขความผิดพลาดที่เกิดขึ้น เป็นต้น

อย่างไรก็ตามไคลเอนต์ ต้องสามารถค้นพบ grid service description ที่เปลี่ยนแปลงที่ไม่ใช่ backward-compatible ได้ ซึ่ง backward-compatible นั้นจะทำให้เกิดการเปลี่ยนรูปแบบหรือการทำของ กริดเซอร์วิสได้

7.5.4.2 ระเบียบการให้ชื่อของ Grid Service Description

ใน WSDL แต่ละ portType จะใช้ถูกใช้ทั่วไปและมีชื่อที่เป็นเอกเทศกันผ่านเงื่อนไขการมีชื่อของมัน ซึ่งจะมีการรวมกันของขอบเขตของชื่อที่บอกด้วย portType และมีค่าชื่อที่เป็นเอกเทศกันในแต่ละ คำ portType ภายในขอบเขตของชื่อนั้นๆ ใน OGSII จะใช้การควบคุมการจัดการโดยอาศัยค่าทั้งหมดของ grid service description ที่เปลี่ยนแปลงไม่ได้ โดยใช้ QName ของ Grid service prtType, operation, message, serviceData และ underlying type จะต้องกำหนดจาก WSDL/XSD ถ้าเปลี่ยนค่านี้ไปแล้ว portType ใหม่นั้นจะต้องกำหนดค่า QName ใหม่ด้วย นั้นหมายความว่า จะมีชื่อภายในใหม่ และมีขอบเขตชื่อใหม่ด้วย

ระหว่างพัฒนาระบบนั้น grid service instance จะสามารถเปลี่ยนแปลงได้ทั้งรูปแบบ, portType หรือแม้กระทั่งระดับการ implement ทำให้ผู้พัฒนาระบบต้องเลือกที่จะใช้งานมันอย่างระมัดระวัง ซึ่งก็ไม่น่ากังวลนักเพราะว่าการเปลี่ยนแปลงจะต้องผ่านการแสดงค่าของ portType ใหม่นั้นอยู่แล้ว

7.5.5 การให้ชื่อกริดเซอร์วิสอินสแตนซ์

แต่ละกริดเซอร์วิสอินสแตนซ์นั้นจะมี อย่างน้อย 1 Grid Service Handles (GSH) ดังที่กล่าวไปช่วงต้น ซึ่ง GHS นั้นจะใช้ชื่อที่อยู่ในรูปแบบของ URI และไม่ได้เป็นส่วนที่เก็บข้อมูลว่าจะยอมให้ client ติดต่อตรงมาที่เซอร์วิสอินสแตนซ์อย่างไร โดยที่ client นั้นจะทราบว่าจะต้องติดต่อกับ เซอร์วิสอินสแตนซ์ไหนนั้นจะต้องผ่านการ resolve ค่า GSH ให้เป็น Grid Service Reference (GSR) ก่อน โดยที่ GSR จะเป็นส่วนที่เก็บข้อมูลที่จำเป็นที่จะใช้ในการติดต่อกับเซอร์วิสอินสแตนซ์ผ่านการผูกของโปรโตคอลในเครือข่ายนั้นๆ

เหมือนกับ URI โดย GSH จะเกิดจาก scheme โดยตามด้วยสตริงที่บรรจุข้อมูลพิเศษ โดย scheme นั้นจะแสดงว่ามันจะแสดงค่าอะไรบ้าง และ resolve จาก GSH ให้เป็น GSR ได้ยังไง โดย client จะเลือกที่จะ resolve GSH ด้วยตนเองหรืออาจจะเลือกโดยอาศัยแหล่งภายนอกก็ได้ เช่น แก้ไขวิธีการทำ HandleResolver ที่ portType เป็นต้น

รูปแบบของ GSR จะเป็นลักษณะพิเศษโดยใช้ผ่าน client เพื่อติดต่อสื่อสารกับกริดเซอร์วิสอินสแตนซ์ เช่น RMI/IIOP จะใช้มัน FSR จะเลือกรูปแบบของ IOR แต่ถ้าใช้ SOAP แล้ว GSR ก็จะเลือกรูปแบบของ WSDL มาใช้

ขณะที่ GSH จะถูกต้องในโลโก้ใหม่ของกริดเซอร์วิสอินสแตนซ์นั้น ถ้า GSR มีการผิดพลาดจะทำให้ไคลเอนต์ต้องการที่จะ resolve ใหม่เพื่อให้ได้ค่า GSR ที่ถูกต้อง

7.5.5.1 Grid Service Reference (GSR)

กริดเซอร์วิสอินสแตนซ์นั้นจะสร้างการเข้าถึงกับแอปพลิเคชันของ client ได้ผ่านการใช้ GSR โดยที่ GSR จะเป็น network-wide pointer เพื่อเข้าสู่กริดเซอร์วิสอินสแตนซ์ที่อยู่ในโฮสต์ที่พร้อมจะทำงาน โดยแอปพลิเคชันของ client นั้นจะสามารถใช้ GSR เพื่อส่งการร้องขอตรงไปที่อินสแตนซ์ที่กำหนดโดย GSR ซึ่ง GSR จะรองรับการเขียนโปรแกรมผ่านกริดเซอร์วิสอินสแตนซ์ที่ชื่อ “by reference” ซึ่ง GSR จะบรรจุข้อมูลที่จำเป็นสำหรับการที่จะเข้าถึงกริดเซอร์วิสอินสแตนซ์ในโฮสต์ได้ผ่านหลายๆโปรโตคอล

การเอนโค้ดของ GSR นั้น บางทีจะได้รับจากหลายรูปแบบในระบบ เหมือนกับกระบวนการอื่นๆในระบบ โดยการเอนโค้ดที่แท้จริงนั้นจะผ่านรูปแบบของ GSR ที่เปรียบเสมือนการผูกผ่านเว็บเซอร์วิสด้วยกริดเซอร์วิสอินสแตนซ์ อย่างที่กำหนดไว้ในรูปแบบการเอนโค้ด WSDL ของ GSR ที่ใช้สำหรับการผูกในบางครั้ง

GSR จะใช้รูปแบบ “on the wire” จากการสร้าง GSR ในกริดเซอร์วิสโฮสต์ เมื่อส่วนที่ใช้อ้างได้ส่งไปที่พารามิเตอร์ที่เป็นของกระบวนการที่ใช้กำหนด WSDL แล้ว แอปพลิเคชันของไคลเอนท์จะส่งผ่านการร้องขอข้อความจาก WSDL-defined operation ซึ่งควรจะรวมทั้งหมดของที่อยู่ที่มีเซอร์วิสที่ต้องการจะสื่อสารนั้นผ่านการสร้างเซอร์วิสอินสแตนซ์ด้วย

ค่าใดๆของ GSR นั้นจะต้องคงอยู่ในระบบ โดยที่วัฏจักรชีวิตของ GSR จะต้องเป็นอิสระกับความเกี่ยวข้องของกริดเซอร์วิสอินสแตนซ์ ซึ่ง GSR จะยังคงถูกต้องตราบเท่าที่ เซอร์วิสอินสแตนซ์ยังคงสามารถใช้ได้ผ่าน GSR นี้

เมื่อ GSR ถูกพบว่าถูกต้องและได้ถูกสร้างจากกริดเซอร์วิสอินสแตนซ์ที่ยังคงอยู่ client จะได้รับค่า GSR ใหม่นั้นผ่านการใช้ GSH ของกริดเซอร์วิสอินสแตนซ์ที่เกี่ยวข้อง โดยที่ GSH จะบรรจุค่า binding-specific ที่ใช้สร้าง GSR ไว้ โดยที่ binding-specific ของ GSR นั้นจะรวมทั้ง เวลาทั้งหมดอายุด้วยที่จะประกาศเมื่อ client ต้องการ GSR นั้น ซึ่ง GSR จะต้องถูกต้องตลอดถึงช่วงเวลานั้น และควรจะผิดพลาดเมื่อเลยค่าเวลานั้น

ตัวอย่างโครง XML ที่ใช้กำหนด GSR ตามนี้

```
targetNamespace = "http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:element name="reference" type="ogsi:ReferenceType"/>

<xsd:complexType name="ReferenceType" abstract="true">
  <xsd:attribute ref="ogsi:goodFrom" use="optional"/>
  <xsd:attribute ref="ogsi:goodUntil" use="optional"/>
</xsd:complexType>
```

7.5.5.1.1 WSDL ที่ใช้เอนโค้ด GSR

เป็นสิ่งที่จำเป็นที่ WSDL จะต้องเอนโค้ด GSR ที่บรรจุข้อมูลที่จำเป็นน้อยที่สุดสำหรับว่ามัน

จะต้องถึงกริดเซอร์วิสอินสแตนซ์นั้นได้อย่างไร โดย WSDL GSR จะมีค่า wsdl:definition ซึ่งบรรจุค่า wsdl:service ไว้และอาจจะมีค่าอื่นๆด้วย ดูได้จากโค้ดนี้

```
targetNamespace = http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:complexType name="WSDLReferenceType">
  <xsd:complexContent>
    <xsd:extension base="ogsi:ReferenceType">
      <xsd:sequence>
        <xsd:any namespace="http://schemas.xmlsoap.org/wsdl/"
          minOccurs="1" maxOccurs="1" processContents="lax"/>
      </xsd:sequence>
    </xsd:extension>
  </xsd:complexContent>
</xsd:complexType>
```

7.5.5.2 Grid Service Handles (GSH)

ต้องมีคุณสมบัติตามนี้

1. GSH ต้องมีค่า URI ที่ถูกต้อง
2. GSH จะต้องใช้ได้อย่างทั่วถึงเมื่อเกิดจากการอ้างกริดเซอร์วิสอินสแตนซ์เดียวกัน และ GSH ต้องไม่อ้างมากกว่า 1 กริดเซอร์วิสอินสแตนซ์
3. กริดเซอร์วิสอินสแตนซ์นั้นจะต้องมีอย่างน้อย 1 GSH
4. กริดเซอร์วิสอินสแตนซ์อาจจะมีหลายๆ GSH ได้ถ้าใช้ URI ที่ไม่เหมือนกัน
5. กริดเซอร์วิสอินสแตนซ์จะต้องไม่มีหลายๆ GSH ถ้าใช้ URI ที่เหมือนกัน
6. ค่าของ gridServiceHandle ของกริดเซอร์วิสอินสแตนซ์จะต้องมีค่าเฉพาะค่า SGH ที่อ้างถึงอินสแตนซ์นั้นเท่านั้น
7. อาจจะมีหลายๆ GSRs ที่อ้างไปที่กริดเซอร์วิสอินสแตนซ์อันเดียวกัน
8. ผลจาก GSH อันเดียวกัน อาจทำให้เกิด GSR ที่ต่างกันได้ โดย resolver จะตอบค่า GSR ที่ต่างกันผ่านค่า GSH เดียวกันที่เวลาต่างกัน และอาจจะส่งค่า GSR ที่ต่างกันไปที่ client ที่ต่างกันด้วย
9. GSH อาจจะไม่สามารถ resolve GSR ได้ ไม่ว่าจะก่อน, ในระหว่าง หรือหลังจากที่กริดเซอร์วิสอินสแตนซ์นั้นยังคงมีชีวิตอยู่ อย่างไรก็ตามโอกาสแบบนี้จะเกิดขึ้นไม่บ่อยนักเนื่องจากรูปแบบของ URI ที่ใช้จะมีคุณภาพที่ค่อนข้างดีพอ
10. ผลจากการ resolve จาก GSH เป็น GSR นั้น บางทีอาจจะเชื่อถือได้หรืออาจเชื่อถือไม่ได้ก็เป็นได้ ขึ้นอยู่กับหลายๆอย่าง เช่น ค่าต่างๆที่ client เช็ตไว้ หรือการจำกัดของโปรโตคอลที่ใช้ resolve

ตัวอย่าง XML ที่ใช้กำหนด GSH

```
targetNamespace = "http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:element name="handle" type="ogsi:HandleType"/>

<xsd:simpleType name="HandleType">
  <xsd:restriction base="xsd:anyURI"/>
</xsd:simpleType>
```

7.5.5.3 เซอร์วิสโลเคเตอร์

เซอร์วิสโลเคเตอร์นั้นเป็นโครงสร้างที่บรรจุค่า GSH, ค่า GSR และค่าอินเตอร์เฟซของ QName (portType) ไว้อย่างน้อย 0 หรือมากกว่านั้น โดยค่า GSHs และ GSRs ทั้งหมดนั้นจะอ้างอิงไปที่กริดเซอร์วิสอินสแตนซ์เดียวกัน และอินเตอร์เฟซทั้งหมดนั้นต้องถูกพัฒนาขึ้นจากกริดเซอร์วิสอินสแตนซ์นั้น โดยโลเคเตอร์จะใช้เพื่อยอมรับ GSH หรือ GSR นั้น โดยการกำหนด XML สำหรับโลเคเตอร์ทำดังนี้

```
targetNamespace = "http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:element name="locator" type="ogsi:LocatorType"/>

<xsd:complexType name="LocatorType">
  <xsd:sequence>
    <xsd:element ref="ogsi:handle"
      minOccurs="0" maxOccurs="unbounded"/>
    <xsd:element ref="ogsi:reference"
      minOccurs="0" maxOccurs="unbounded"/>
    <xsd:element name="interface" type="QName"
      minOccurs="0" maxOccurs="unbounded"/>
  </xsd:sequence>
</xsd:complexType>
```

7.5.6 วัฏจักรชีวิตของกริดเซอร์วิส

วัฏจักรชีวิตของทุกๆกริดเซอร์วิสอินสแตนซ์นั้นจะกำหนดโดยการสร้างและการทำลายของ เซอร์วิสอินสแตนซ์นั้นๆ โดยวิธีที่แท้จริงของกริดเซอร์วิสอินสแตนซ์จะถูกสร้างหรือถูกทำลายโดยคุณสมบัติขั้นพื้นฐานของกลุ่มโฮสต์นั้นๆ แต่กระนั้นก็ยังมีความหมายของ portTypes ที่เกี่ยวข้องกันได้กำหนดว่าไคลเอนท์จะติดต่อกับเหตุการณ์ในวัฏจักรชีวิตนี้ได้อย่างไร โดยจะแบ่งเป็นเหตุการณ์ย่อยๆดังนี้

- ไคลเอนท์จะร้องขอเพื่อสร้าง (creation) กริดเซอร์วิสอินสแตนซ์โดยการเรียก createService ของเซอร์วิสอินสแตนซ์ที่จะสร้างขึ้นโดย Factory portType หรือ จากวิธีพิเศษที่ใช้กำหนด เซอร์วิสใหม่ๆ เช่น NotificationSource portType ซึ่งเป็นเซอร์วิสของ factory ที่จะกล่าวถึงต่อไป
- ไคลเอนท์จะร้องขอเพื่อทำลาย (destruction) กริดเซอร์วิสอินสแตนซ์ ผ่านไคลเอนท์ใดๆ โดย explicit destruction operation ที่รองรับโดย GridService portType ซึ่งไคลเอนท์ที่ลงทะเบียนนั้นจะสนใจกับกริดเซอร์วิสอินสแตนซ์เพียงช่วงเวลาหนึ่ง จนกระทั่งเวลานั้นหมดไป โดยที่เซอร์วิสอินสแตนซ์นั้นจะต้องถูกทำลายโดยอัตโนมัติ

กริดเซอร์วิสอินสแตนซ์จะรองรับการจัดการไลฟ์ไทม์แบบซอฟต์สเตต (Soft state) ในกรณีที่ไคลเอนต์จะเริ่มค่าไลฟ์ไทม์นั้นเมื่อกริดเซอร์วิสอินสแตนซ์ได้ถูกสร้างขึ้นผ่าน factory และได้รับการยืนยันจากข้อความ requestTerminationBefore/After (“keepalive”) เพื่อร้องขอที่จะรับไลฟ์ไทม์ของเซอร์วิสนั้น ถ้าถึงเวลาสิ้นสุดแล้ว ฟังก์ชันเวิร์กที่เก็บเซอร์วิสอินสแตนซ์นั้นก็จะทำลาย และรีเซ็ตที่เกี่ยวข้องทั้งหมดทิ้งไป

ซึ่งเวลาสิ้นสุดนั้นจะเปลี่ยนอย่างไม่ซ้ำกัน การที่ไคลเอนต์ร้องขอเวลาสิ้นสุดนั้นจะเร็วกว่าขอเวลาในตอนปัจจุบัน นั่นคือ ถ้ามีการร้องขอเวลาสิ้นสุดโดยที่เวลานั้นเป็นก่อนเวลาปัจจุบัน นั่นคือจำเป็นต้องแสดง สิทธิเพื่อยกเลิกในครั้งนั้นๆเป็นพิเศษ เวลาสิ้นสุดที่แสดงผ่าน OGSi ดูได้ ดังตัวอย่างนี้

```
targetNamespace = "http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:complexType name="TerminationTimeType">
  <xsd:attribute name="after" type="ogsi:ExtendedDateTimeType"
    use="optional"/>
  <xsd:attribute name="before" type="ogsi:ExtendedDateTimeType"
    use="optional"/>
  <xsd:attribute name="timestamp" type="xsd:dateTime"
    use="optional"/>
</xsd:complexType>

<xsd:simpleType name="ExtendedDateTimeType">
  <xsd:union memberTypes="ogsi:InfinityType xsd:dateTime"/>
</xsd:simpleType>

<xsd:simpleType name="InfinityType">
  <xsd:restriction base="string">
    <xsd:enumeration value="infinity"/>
  </xsd:restriction>
</xsd:simpleType>
```

โดยค่า after ของ TerminationTimeType นั้นจะบรรจุเวลาแรกสุดที่เซอร์วิสอินสแตนซ์วางไว้ว่าจะยกเลิก โดยมีค่าพิเศษคือ ค่าอนันต์ ที่ทำให้เซอร์วิสอินสแตนซ์นั้นจะคงอยู่อย่างไม่มีกำหนด

โดยค่า before ของ TerminationTimeType นั้นจะบรรจุเวลาสุดท้ายที่เซอร์วิสอินสแตนซ์วางไว้ว่าจะยกเลิก โดยมีค่าพิเศษคือ ค่าอนันต์ ที่ทำให้เซอร์วิสอินสแตนซ์นั้นจะคงอยู่โดยไม่มีการยกเลิก

โดยค่า timestamp ของ TerminationTimeType จะบรรจุเวลาที่เซอร์วิสอินสแตนซ์ได้เลือกไว้ว่าเวลาที่หลังจากและก่อนหน้าที่กำหนดไว้จะยังคงถูกต้องอยู่ โดย timestamp นั้นจะใช้ใน TerminationTimeType เพื่อใช้เป็นเกณฑ์ของเวลาที่จะเกี่ยวข้องกับ client หรือแหล่งเวลาอื่นๆ

7.5.7 การจัดการเมื่อเกิดกระบวนการผิดพลาด

OGSI จะกำหนด XSD ไว้สำหรับความผิดพลาดทั้งหมดที่กริดเซอร์วิสส่งกลับมาเพื่อใช้แก้ไข ปัญหาต่างๆ โดยบรรจุข้อความที่ผิดพลาดต่างๆไว้ โดย ogsi:FaultType XSD ดังนี้

```

targetNamespace = "http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:element name="fault" type="FaultType">
<xsd:complexType name="FaultType">
  <xsd:sequence>
    <xsd:element name="description"
      type="xsd:string"
      minOccurs="0" maxOccurs="unbounded"/>
    <xsd:element name="originator"
      type="ogsi:LocatorType"
      minOccurs="1" maxOccurs="1"/>
    <xsd:element name="timestamp"
      type="xsd:dateTime"
      minOccurs="1" maxOccurs="1"/>
    <xsd:element name="faultcause"
      type="ogsi:FaultType"
      minOccurs="0" maxOccurs="unbounded"/>
    <xsd:element name="faultcode"
      type="ogsi:FaultCodeType"
      minOccurs="0" maxOccurs="1"/>
    <xsd:element name="extension"
      type="ogsi:ExtensibilityType"
      minOccurs="0" maxOccurs="1"/>
  </xsd:sequence>
</xsd:complexType>

<xsd:complexType name="FaultCodeType">
  <xsd:simpleContent>
    <xsd:extension base="xsd:string">
      <xsd:attribute name="faultscheme" type="anyURI"
        use="required"/>
    </xsd:extension>
  </xsd:simpleContent>
</xsd:complexType>

<xsd:complexType name="ExtensibilityType">
  <xsd:sequence>
    <xsd:any namespace="##any"/>
  </xsd:sequence>
</xsd:complexType>

```

โดยค่าความผิดพลาดทั้งหมดจากกริดเซอร์วิสอินสแตนซ์นั้นจะใช้ FaultType ที่แก้ไขต่างกัน โดยค่าต่างๆที่ใช้ใน OGSI portTypes จะใช้ดังนี้

ค่า description จะบรรจุรายละเอียดความผิดพลาดเป็นภาษาตรงๆ กล่าวคือ เป็นส่วนอธิบายความผิดพลาดกับผู้ใช้งาน ค่านี้เป็นแบบ optional ไม่จำเป็นต้องมีก็ได้

ค่า originator เป็นตัวบอกตำแหน่งของเซอร์วิสอินสแตนซ์ที่ผิดพลาด

ค่า timestamp เป็นค่าเวลาที่ความผิดพลาดเกิดขึ้น

ค่า faultcause เป็นค่าใน ogsi:FaultType ใช้อธิบายผลลัพธ์ที่ผิดพลาด ใช้ร่วมกับ xsi:type ใช้เพื่อให้ทราบรายละเอียดเหตุผลของความผิดพลาด

ค่า faultcode ใช้ในการรายงานความผิดพลาดให้ระบบ โดยอาศัย faultscheme ที่แสดงเป็น URI

ค่า extension ใช้ระบุข้อความพิเศษที่เกี่ยวกับความผิดพลาดในรูปแบบขอบเขตชื่อของ XML ซึ่งบางทีจะเป็นค่าตัวเลขของค่าที่สืบเนื่องมา

โดยค่าความผิดพลาดทั้งหมดนั้นจะตอบกลับในรูปแบบ ogsci:fault ในการแจ้งความผิดพลาดดังนี้

```
<wsdl:definitions ...>
  <types>
    <xsd:schema ...>
      <xsd:complexType name="MyFaultType">
        <xsd:complexContent>
          <xsd:extension base="ogsci:FaultType"/>
        </xsd:complexContent>
      </xsd:complexType>
      <xsd:element name="myFault" type="tns:MyFaultType"/>
    </xsd:schema>
  </types>

  <message name="myFaultMessage">
    <part name="fault" element="tns:myFault"/>
  </message>

  <gwsdl:portType ...>
    <wsdl:operation ...>
      <input ...>
      <output ...>
      <fault name="myFault" message="tns:myFaultMessage"/>
      <fault name="fault" message="ogsci:faultMessage"/>
    </wsdl:operation>
  </gwsdl:portType>
</wsdl:definitions>
```

7.5.8 กระบวนการที่ใช้ขยาย

หลายๆกระบวนการใน OGSI จะรับค่าที่ไม่มีรูปแบบสืบทอดกันมา เช่น รับค่าที่ให้ไคลเอนท์ค้นหากระบวนการที่ถูกต้องเป็นต้น คุณได้จากส่วนที่เป็น NotificationSource::subscribe ที่ใช้เพื่อให้ไคลเอนท์ถามหาเซอร์วิสอินสแตนซ์ที่ต้องการใช้ ของ serviceDataValues ที่เปลี่ยน อย่างไรก็ตามเซอร์วิสที่ใช้ทำ portType ที่สืบทอดมาจาก NotificationSource นั้นจะแสดงกำหนดโดยรูปแบบการคิวรีซึ่งมีประสิทธิภาพมากกว่า โดยส่วนนี้จะอธิบายถึงไคลเอนท์จะเลือกการลงท้ายจาก NotificationSource portType ได้อย่างไร คุณได้จากตัวอย่างข้างล่างนี้

```
targetNamespace = "http://www.gridforum.org/namespaces/2003/03/OGSI"

<xsd:complexType name="OperationExtensibilityType">
  <xsd:attribute name="inputElement" type="QName" use="optional"/>
</xsd:complexType>
```

แต่ละ portType จะมีการกำหนด serviceData ของ OperationExtensibilityType และมีค่า mutability="static" โดยค่า static ของ SDE นี้จะกำหนดด้วยค่าที่สืบทอดอย่างถูกต้อง ดังตัวอย่างเช่น มี portType ชื่อ myPT ซึ่งมี myOperation อยู่ ซึ่งจะสร้างวิธีเรียกค่าออฟท์ได้ 2 แบบ คือ

myop:myOption1 และ myop:myOption2 ที่ผ่านเข้าไปสู่ส่วนที่สืบทอดจาก myOperation ได้ โดยการกำหนด myOperation ทำได้ดังนี้

```
<gwsdl:portType name="myPT">
  ...
  <sd:serviceData name="myOperationExtensibility"
    type="ogsi:OperationExtensibilityType"
    minOccurs=0 maxOccurs="unbounded"
    mutability="static"
    modifiability="false"
    nillable="false" />
  ...
  <sd:staticServiceDataValues>
    <myOperationExtensibility inputElement="m1:myOption1"/>
    <myOperationExtensibility inputElement="m1:myOption2"/>
  </sd:staticServiceDataValues>
  ...
</gwsdl:portType>
```

โดยค่า inputElement ของ SDE จะเป็นค่า QName ที่เป็นเอกเทศกัน โดยมาจากการประกาศของค่า XSD ที่กำหนดที่ค่าที่ถูกต้อง โดยคุณสมบัติทั้งหมดจะถูกแสดงโดย inputElement ของ OperationExtensibilityType เช่น inputElement ที่แสดงโดยค่าพารามิเตอร์จากกระบวนการที่แสดงว่ากระบวนการนั้นจะรับค่า inputElement นั้นได้อย่างไร

ถ้า inputElement นั้นถูกละเว้นจากค่า SDE แล้ว มันจะต้องถูกแสดงด้วยการขยายค่าที่ผ่านการกระบวนการเรียกนั้น

ตัวอย่างกระบวนการที่รวมค่า portTypes ที่ขยายจากการประกาศ serviceData เช่น มี portType ชื่อ myPT2 ที่ขยาย myPT จะกำหนดค่าที่ถูกต้องให้กับ myOperation ตามนี้

```
<gwsdl:portType name="myPT2" extends="m1:myPT">
  ...
  <sd:staticServiceDataValues>
    <myOperationExtensibility inputElement="m2:myOption3"/>
    <myOperationExtensibility/>
  </sd:staticServiceDataValues>
  ...
</gwsdl:portType>
```

ในตัวอย่างนี้ เซอร์วิสอินสแตนซ์ที่ใช้ทำ myPT2 จะรองรับการขยายอินพุทของ myOperation: m1:myOption1, m1:myOption2, m2:myOption3 และ ไม่มีค่าที่ทั้งหมด โดยแต่ละกระบวนการจะเกี่ยวข้องกันกับ inputElement

7.6 อินเทอร์เฟซของกริดเซอร์วิส

ส่วนนี้เป็นส่วนที่ระบุเกี่ยวกับการรวมตัวของ WSDL portTypes และพฤติกรรมที่เกี่ยวข้อง

ทั้งหมด โดยเป็นกลุ่มฟังก์ชันพื้นฐานของแนวทางการคิดแบบกระจาย โดยจะอธิบายในตารางด้านล่าง และจะแยกออกเป็นหัวข้อย่อย ในส่วนถัดไป

portType Name	Description
GridService	เป็นส่วนที่แสดงรูปแบบของเซอร์วิสนั้นๆ
HandleResolver	ทำการโยงจาก GSH ให้เป็น GSR
NotificationSource	ยอมให้ client ลงท้ายข้อความประกาศ
NotificationSubscription	กำหนดความสัมพันธ์ระหว่าง NotificationSource กับ NotificationSink
NotificationSink	กำหนดกระบวนการส่งข้อความประกาศไปที่เซอร์วิสอินสแตนซ์
Factory	เป็นกระบวนการมาตรฐานที่ใช้สร้าง กริดเซอร์วิสอินสแตนซ์
ServiceGroup	ยอมให้ client ดูแกล่มของเซอร์วิส
ServiceGroupRegistration	ยอมให้กริดเซอร์วิสเพิ่มหรือลดออกจาก ServiceGroup
ServiceGroupEntry	กำหนดความสัมพันธ์ระหว่างกริดเซอร์วิสอินสแตนซ์กับสมาชิกภายใน ServiceGroup

ตารางที่ 7-3 แสดงอินเตอร์เฟสของกริดเซอร์วิส

ซึ่ง OGSI portType ทั้งหมดนี้กำหนดในขอบเขตชื่อของ OGSI

7.6.1 GridService portType

GridService portType จะเป็นส่วนที่ใช้สร้างกริดเซอร์วิสทั้งหมดและใช้จำกัดความพื้นฐานใน OGSI โดยลักษณะของการใช้ portType นั้นจะคล้ายคลึงกันกับการใช้อ็อบเจกต์คลาส เหมือนใน object-oriented programming language เช่น smalltalk หรือ Java ซึ่งจะซ่อนส่วนต่างๆเป็นองค์ประกอบย่อยๆ โดย GridService portType นั้นจะใช้ในการ คิวรีและอัปเดต serviceData ของกริดเซอร์วิสอินสแตนซ์ และใช้จัดการการยกเลิกอินสแตนซ์

ในรูปแบบของเว็บเซอร์วิสนั้น จะเลือกวิธีอย่างใดอย่างหนึ่งระหว่าง ใช้รูปแบบการส่งเอกสาร หรือ ใช้ remote procedure call (RPC) แต่กริดเซอร์วิสนั้นได้ถูกออกแบบให้อิสระ สามารถใช้งานทั้ง 2 รูปแบบผสมผสานกันได้

7.6.2 HandleResolver PortType

HandleResolver portType นั้นเป็นส่วนที่ใช้ resolve ค่า GSH ให้เป็น GSR โดยเป็นอิสระกับ URI ใดๆของ GSH ซึ่งค่าเซอร์วิสอินสแตนซ์นั้นจะผ่าน HandleResolver portType ที่เรียกว่า handle resolver

แต่ละ handle resolver นั้นจะมีวิธีการที่ต่างกันเกี่ยวกับการแม็พจาก GSH ให้เป็น GSR โดยบาง handle resolver นั้นจะใช้เซอวิซของส่วนที่จัดการเกี่ยวกับไลฟ์ไทม์ของ hosting environment มาใช้โดยตรง เช่น การสร้างและการทำลายอินสแตนซ์แบบอัตโนมัติ หรือ การเพิ่มและลบการแม็พ เมื่อเซอวิซอินสแตนซ์นั้นลงทะเบียว่าคงอยู่กับ resolver แล้ว resolver จะคิวรีค่า gridServiceHandle และค่า gridServiceReference ของอินสแตนซ์นั้นเพื่อแม็พลงฐานข้อมูล

portType นี้จะสืบเนื่องมาจาก GridService portType

7.6.3 Notification

จุดประสงค์ของ notification คือส่งข้อความที่น่าสนใจจาก notification source ไปที่ notification sink โดยจะอธิบายส่วนต่างๆ ไว้ตามนี้

- notification source คือ กริดเซอวิซอินสแตนซ์ที่ใช้ทำ NotificationSource portType และเป็นผู้ส่ง notification messages โดยที่แหล่งข้อมูลนั้นอาจจะส่ง notification message ไปที่ sink ใดๆก็ได้
- notification sink เป็นกริดเซอวิซอินสแตนซ์ที่ได้รับ notification message จากแหล่งข้อมูลใดๆ โดย sink จะใช้ทำ NotificationSink portType ซึ่งยอมให้รับค่า notification message
- notification message เป็นค่า XML ที่ส่งจาก notification source ไปยัง notification sink โดยค่า XML นี้จะถูกเลือกโดย subscription expression
- subscription expression เป็นค่า XML ที่ใช้อธิบายว่าข้อความนั้นควรจะส่งจาก notification source ไปยัง notification sink โดยขึ้นอยู่กับค่า serviceDataValues ของเซอวิซอินสแตนซ์
- ในการหาว่า notification message อะไรจะต้องถูกส่งไปที่ไหนนั้น จะใช้การร้องขอค่า subscription ไปที่แหล่งข้อมูล ประกอบไปด้วย subscription expression, locator ของ notification sink ที่ notification message จะส่งไป และค่าเริ่มต้นของเวลาชีวิตสำหรับ subscription
- การร้องขอ subscription นั้นจะทำให้เกิดการสร้าง กริดเซอวิซอินสแตนซ์ ที่เรียกว่า subscription ที่ใช้สร้าง NotificationSubscription portType โดย portType นี้จะถูกใช้โดยไคลเอนท์เพื่อจัดการไลฟ์ไทม์ของ subscription และค้นหาคุณสมบัติของค่า subscription

7.6.3.1 NotificationSource PortType

Notification portType จะยอมให้ client เพื่อลงท้าย notification message จากกริดเซอวิซอินสแตนซ์ที่ใช้ทำ portType นี้ โดยที่ NotificationSource portType จะขยายมาจาก GridService portType

7.6.3.2 NotificationSink portType

NotificationSink portType นั้นกำหนดโดยคำสั่งเดียวที่ใช้สำหรับส่ง notification message ไปให้เซอวิซอินสแตนซ์ที่ใช้ทำคำสั่งนั้น

7.6.4 Factory PortType

factory เป็นรูปแบบที่ตรงตามกริดเซอร์วิสที่ไคลเอนต์สร้างขึ้นโดยกริดเซอร์วิสอินสแตนซ์อื่นๆ โดยที่ไคลเอนต์จะเรียกสร้างคำสั่งบน factory และได้รับผลตอบรับเป็น locator ของเซอร์วิสอินสแตนซ์ใหม่ที่ถูกสร้างขึ้น โดย factory อาจเป็นกริดเซอร์วิสอินสแตนซ์ที่ใช้สร้าง Factory portType หรืออาจเป็นกริดเซอร์วิสอินสแตนซ์ที่สร้างคำสั่งของ factory โดยเฉพาะ เช่น NotificationSource::subscribe เป็นคำสั่งที่อธิบายไปในเรื่องที่แล้ว

บนการสร้าง factory นั้น กริดเซอร์วิสอินสแตนซ์ที่ถูกสร้างขึ้นใหม่นั้นจะถูกลงทะเบียนและได้รับค่า GSH ซึ่งเป็น handle resolution service ที่ใช้ในการลงทะเบียนกับ hosting environment

ซึ่ง Factory portType จะต้องสืบเนื่องมาจาก GridService portType

7.6.5 ServiceGroup portType

ServiceGroup เป็นกริดเซอร์วิสอินสแตนซ์ที่ดูแลข้อมูลข่าวสารเกี่ยวกับกลุ่มของกริดเซอร์วิสอื่นๆ เช่น กริดเซอร์วิสที่ใช้ลงทะเบียนจะกำหนดโดย portType ที่สืบเนื่องมาจากพฤติกรรมพื้นฐานจาก ServiceGroup ซึ่งมี 3 portType ที่ดูแลรูปแบบของกลุ่มของเซอร์วิสนี้ คือ ServiceGroup, ServiceGroupEntry และ ServiceGroupRegistration

ServiceGroup portType เป็นส่วนที่ใช้จัดเตรียมรูปแบบการแทน service group โดยมีอย่างน้อย 0 เซอร์วิสที่เป็นสมาชิกในกลุ่ม โดยค่า SDE ของ ServiceGroup นั้นจะประกอบไปด้วยค่าสำหรับแต่ละกริดเซอร์วิสที่เป็นสมาชิกในกลุ่ม โดยมีค่า ServiceGroupEntryLocator ใช้อ้างอิงที่เซอร์วิสอินสแตนซ์ที่ใช้ทำ ServiceGroupEntry portType ซึ่งเตรียมฟังก์ชันควบคุมการดูแลไว้ โดยจะมีค่าที่เป็นเอกเทศไว้ใช้เป็นคีย์คือ GSH สำหรับแต่ละค่า

คุณสมบัติต่อไปนี้จะเป็นคุณสมบัติที่จำเป็นของ ServiceGroup, ServiceGroupEntry, ServiceGroupRegistration และ สมาชิกของกริดเซอร์วิสอินสแตนซ์ของ ServiceGroup

- กริดเซอร์วิสอินสแตนซ์จะต้องมีสมาชิกของหลายๆ ServiceGroups
- สมาชิกของกริดเซอร์วิสอินสแตนซ์ของ ServiceGroup จะต้องทำในหลายๆ portType ที่ต่างกัน
- ค่า ServiceGroupEntry จะต้องถอดออกจาก ServiceGroup โดยการจัดการไลฟ์ไทม์ของ ServiceGroupEntry
- เมื่อ ServiceGroup ถูกทำลาย client จะต้องไม่ได้รับผลกระทบจากการทำลายเซอร์วิส ServiceGroupEntry นั้น
- เมื่อ ServiceGroup ถูกทำลาย client จะต้องสามารถทราบเกี่ยวกับการไม่อยู่ของ ServiceGroupEntry หรือรายละเอียดที่ถูกต้องของมัน (เช่น คุณสมบัติของไลฟ์ไทม์)
- ServiceGroupEntry จะต้องเป็นของหนึ่ง ServiceGroup

- ถ้า GridService รวมค่าการยกเลิก ServiceGroup ไว้ด้วยแล้ว ServiceGroup จะต้องไม่สนใจมัน
- สมาชิกทั้งหมดของกริดเซอร์วิสอินสแตนซ์ของ ServiceGroup จะต้องทำตามกันอย่างน้อย 1 portType ที่เป็นสมาชิกใน membershipContentRule ของ ServiceGroup
- กริดเซอร์วิสอินสแตนซ์ที่เป็นสมาชิกจะต้องรวมอยู่ใน ServiceGroup โดยที่ ServiceGroup เปรียบเสมือนถุงที่รวบรวมค่าต่างๆไว้ โดยผู้ออกแบบได้ใช้สืบนี้อิงมาจาก ServiceGroup ซึ่งเป็นเซตของกลุ่มต่างๆ